Planul proiectului, feature cu feature

Spring Boot 4.1 · Java 25 · MySQL 8.4 · Flyway · Spring AI 2.0.2 (MCP) · Kafka · Prometheus + Grafana · ELK + Tempo · Keycloak · Kong · nginx · React + Vite + TypeScript · Playwright · GitHub Actions · doar Docker (compose) pe EC2 · Flavius

Pornești de la codul tău, nu de la zero. Azi există doar `Categories` : GET și POST pe rutele din contract, 15/15 teste verzi; mai rămâne lista goală, care trebuie să fie `[]` , nu 404 (F1). Stilul tău rămâne: Command/Query separate și mapper static. Query service-urile tale devin exact baza tool-urilor MCP din F9. Modelele `User` , `Provider` , `Offering` , `Slot` și `Booking` au fost șterse: le rescrii pe rând în F2–F6, după `STRUCTURA_BD.pdf` , nu le recuperezi din istoric.
Înlocuiește `PLAN_BACKEND.pdf` și `FEATURES.pdf` . Tabelele rămân în `STRUCTURA_BD.pdf` , contractul REST în `openapi.yaml` (v1.2.0, 21 de operații). Bug-urile din `CODE_REVIEW.pdf` se repară **în feature-ul căruia îi aparțin**: le găsești la „Repară”.
Scopul: să construiești singur, pe slot-mate (rezervări de intervale la frizeri, dentiști, săli), tot drumul unui sistem real: un flux fără om în față, metrici, server MCP, un agent care propune, un asistent Spring AI pentru clienți, Kafka, un serviciu desprins, tracing cu loguri în ELK, apoi platforma „ca la cars”: login central în Keycloak și Kong ca singura poartă spre aplicații. Lângă ele, interfața pe care o folosesc oamenii: `slot-mate-client` , în React, crescută în pași U1–U5, fiecare făcut imediat după feature-ul de backend de care are nevoie (partea V). Doar Docker: `docker compose` cu profiluri, niciodată Kubernetes. Fiecare unealtă intră atunci când rezolvă o problemă a domeniului.
Fiecare feature merge de la A la Z, în aceeași ordine:

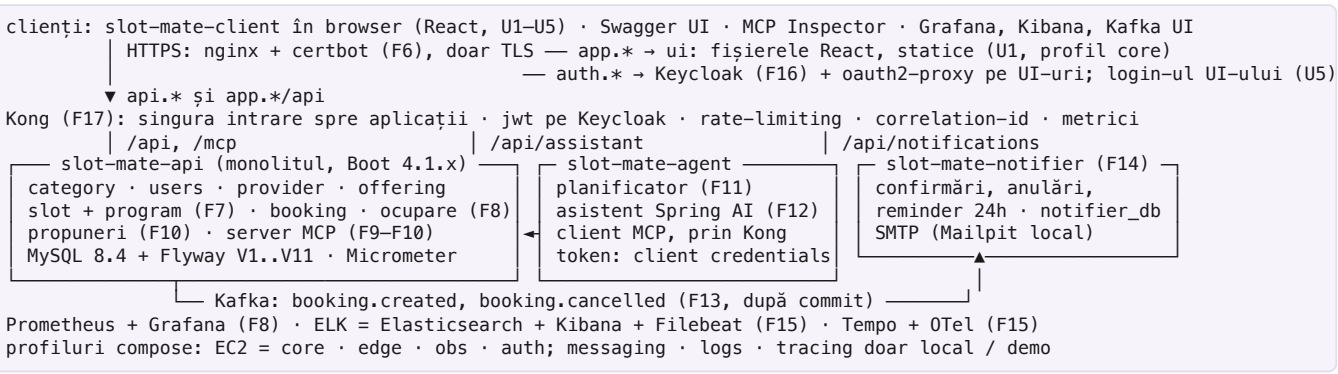
ce poate face utilizatorul → tabela (din entitate până la F5, migrare `Vn__…sql` din F5c) → model + repository → service → intrare (REST, tool MCP sau Kafka) → permisiuni → test cu login real → log / metrică → teste verzi (local până la F5, în Actions din F5b) → verificat pe EC2
Un feature e gata doar când trece „Gata când” **pe EC2, cu token din login**: `/api/auth/login` până la F15, Keycloak din F16 (F1 și F5b sunt singurele fără token; F13–F15 se verifică local și pe instanța de demo, F16–F17 pe EC2, plus demo pentru Kibana și Kafka UI). U1–U5 la fel: pe EC2, în browser, cu login real. Ordinea completă: F1–F5, F5b, F5c, **U1**, F6, **U2**, F7–F10, **U3**, F11, F12, **U4**, F13–F16, **U5**, F17. Nu începi următorul înainte.
Fiecare feature aduce **cel mult o unealtă nouă**, marcată **nou** .

F1–F5 le faci singur, doar cu ce știi deja. Niciunul nu are **nou** :

- **Spring Boot + Data JPA:** entități, repository-uri, tabelele create din entități (`ddl-auto: update`), ca la lecția de Spring Data JPA.
- **Spring Security:** BCrypt, JWT cu `jjwt` , filtrul tău (`OncePerRequestFilter`), `SecurityFilterChain` , `@EnableMethodSecurity` , ca la lecția de Spring Security.
- **Teste:** JUnit + Mockito pe service-uri, ca la lecția de testare. Testele de integrare folosesc Testcontainers-ul deja pus în proiect: îți Docker pornit și rulezi `./gradlew test` înainte de fiecare push.
- **Livrare de mână, ca în DevOps L1-2…L1-5:** Dockerfile, imaginea pe Docker Hub, `docker-compose.yml` , `scp` + `ssh` pe EC2. La fiecare feature: imagine nouă, apoi `docker compose pull && docker compose up -d` pe EC2.

Uneltele noi încep după F5, câte una pe feature, fiecare cu lecția ei înainte: GitHub Actions + GHCR (F5b), Flyway (F5c), React (U1), nginx + certbot (F6), apoi restul traseului.

Unde ajungi



Cine ce poate — enum-ul complet, de la F2

Permisiune	vizitator	CLIENT	PROVIDER	ADMIN	AGENT	Folosită în
`CATEGORY_MANAGE`	–	–	–	✓	–	F2 (în F1, POST e deschis)
`PROVIDER_PROFILE_MANAGE`	–	–	profilul lui	–	–	F3
`OFFERING_MANAGE`	–	–	serviciile lui	–	–	F4
`SLOT_MANAGE`	–	–	intervalele și programul lui	–	–	F5, F7
`BOOKING_CREATE`	–	✓	–	–	–	F6, F12
`BOOKING_VIEW`	–	ale lui	la firma lui	–	–	F6, F9
`BOOKING_CANCEL`	–	ale lui	la firma lui	–	–	F6
`STATS_VIEW`	–	–	firma lui	toți	toți	F8, F9, F11
`SLOT_PROPOSE`	–	–	–	–	✓	F10, F11
`SLOT_PROPOSAL_VIEW`	–	–	pentru firma lui	toate	rulările lui	F10, F11
`SLOT_PROPOSAL_APPROVE`	–	–	pentru firma lui	–	–	F10
`ASSISTANT_USE`	–	✓	–	–	–	F12
`NOTIFICATION_VIEW`	–	ale lui	ale lui	–	–	F13, F14

„al lui” / „firma lui” = permisiunea e în token, iar service-ul verifică proprietarul, la fel pentru REST și MCP. `AGENT` nu are `SLOT_MANAGE` și nici `SLOT_PROPOSAL_APPROVE` : agentul doar **propune**, nu publică și nu se aprobă singur (F10). **Sursa rolurilor se schimbă o singură dată.** F2–F15: `users.role` + harta `RolePermissions` din API, puse în claim-ul `permissions` la login. Din F16: `realm-slot-mate.json` din Keycloak (fiecare permisiune = rol de realm, `CLIENT` / `PROVIDER` / `ADMIN` / `AGENT` = roluri compuse, exact tabelul de sus), puse de Keycloak în acelasi claim `permissions` ; coloana `AGENT` devine contul de serviciu al clientului `slot-mate-agent` . Tabelul nu se schimbă, iar `hasAuthority` nici atât.
Vizitatorul (fără token) are doar rutele publice: register, login (la Keycloak din F16), `GET /api/categories` , `GET /api/providers` , `GET /api/providers/{providerId}` , `GET /api/providers/{providerId}/offerings` , `GET /api/offerings/{offeringId}/slots` , Swagger UI (`/swagger-ui.html` , `/swagger-ui/**` , `/v3/api-docs/**`), `/error` și `/actuator/health` + `/actuator/prometheus` (doar pe portul de management 8081, de la F8).

I. Domeniul merge (monolit)

O singură aplicație, `slot-mate-api`. La final ai contractul întreg pe EC2, intervalele publicate singure noaptea și ocuparea live în Grafana.

F1 Categoriile, până pe EC2

vizitator · (administrator, de la F2)

Oricine vede lista categoriilor ordonată după nume (Barber, Frizerie, Sala de sport…), iar fără categorii lista e `[]`, nu eroare. Adăugarea unei categorii merge deja; de la F2 o face doar administratorul.

Pornești de la: branch-ul `feat/implement-feature-1`: `CategoryController`, `CategoryQueryServiceImpl.getAllCategories`, `CategoryCommandServiceImpl`, `CategoryRepository`, `CategoryMapper`, `GlobalExceptionHandler`, `CategoryNotFoundException`, `Category`, `application.yaml`, `build.gradle`, `TestcontainersConfiguration`, testele tale și ale profesorului, `Dockerfile`, `slot-mate-api/docker-compose-slot-mate-api.yaml` · **Repară (runda 2): B2 lista goală, M4, C1–C7** · fără unealtă nouă

1. **Bază.** Tabela `categories` o face Hibernate din entitate (`ddl-auto: update`), ca până acum. Scoți `show-sql` și dialectul explicit (C7). Migrările vin abia în F5c.
2. **Model/repository.** `Category` rămâne cum e. În `CategoryRepository`: `findAllByOrderByNameAsc()`.
3. **Service.** `getAllCategories` întoarce lista ordonată după nume, iar pe tabel gol `[]` (B2). `CategoryNotFoundException` și constanta ei dispar: nu mai e nimic de mapat. Fără dubla căutare după nume, cu `orElseThrow` (M4); fără `throws` inutil în interfață (C4). `@Transactional` din `org.springframework.transaction.annotation`, cu `readOnly` pe query (C1).
4. **Intrare.** `GET /api/categories`, fără `/all`, cu corp array; dacă `CategoryResponseList` mai există, dispare. Câmpurile controller-ului devin `final`, log-urile au exact câte `{}` au argumente (C2), fără importuri nefolosite (C3). Un bean `OpenAPI` cu `info.version = 1.2.0`.
5. **Config.** `PORT` implicit trece de la 8085 la 8080, ca în contract și compose. Toolchain-ul Java 25 rămâne; în documente, 21 devine 25.
6. **Permisiuni.** Niciuna încă: POST rămâne deschis până la F2, cum spune contractul.
7. **Test (primul scris de tine).** În `CategoryIntegrationTest`: GET pe bază goală → 200 `[]`; trei POST-uri în ordine amestecată → GET le dă ordonate; „barber" după „Barber" → 409. `CategoryQueryServiceTest` îl rescrii pe `List<CategoryResponse>`, fără `CategoryResponseList` și `CategoryNotFoundException` (altfel nu compilează): `getCategoriesReturnEmptyList` verifică `[]`, nu excepția. Toate testele cu `// Arrange / Act / Assert` și doar `assertThat` (C5).
8. **Livrare (ca în L1-2…L1-5 din DevOps).** `slot-mate-api/Dockerfile` în două etape (varianta Gradle: gradle → `eclipse-temurin:25-jre`), imaginea împinsă de mână pe Docker Hub, pentru arhitectura instanței EC2. Fără comentarii în `Dockerfile` (C6). Compose-ul tău din `slot-mate-api/` îl muți în `slot-mate-ops/docker-compose.yml`: `mysql:8.4` (volum + healthcheck) și api cu imaginea de pe Docker Hub, nu `build: .` (`service_healthy`, port 8080); în repo intră doar `.env.example`. `scp` pe compose, apoi `ssh + docker compose pull && docker compose up -d`; `.env` pus o singură dată pe EC2, de mână. Security Group: 22 și 8080. Așa livrezi și F2–F5.
9. **Merge.** Când „Gata când" trece, faci merge pe `master`. Conflictul pe `CODE_REVIEW.pdf` (șters pe `master`, modificat pe branch) îl rezolvi păstrând versiunea din branch: `git add CODE_REVIEW.pdf`, apoi commit.

Reguli

- Nume obligatoriu, maxim 50 de caractere → 400 cu `errors.name`. Nume unic, fără să conteze literele mari și mici → 409 ProblemDetail.
- GET întoarce un array ordonat după nume; fără categorii → 200 `[]`, niciodată excepție. Singurul feature fără token: conturile vin în F2.

Gata când: pe EC2: `GET http://<ip>:8080/api/categories` pe bază goală → 200 `[]` · POST „Sala de sport", „Frizerie", „Barber" → 3 × 201 · POST „barber" → 409 · GET → 3 elemente, în ordinea Barber, Frizerie, Sala de sport · `/api/categories/all` → 404 · `/v3/api-docs` → `info.version = 1.2.0`.

F2 Contul meu

vizitator · orice utilizator · administrator

Vizitatorul își face cont de client sau de furnizor, se autentifică și primește un token valabil o oră; cu el își vede contul. De acum doar administratorul adaugă categorii.

Pornesti de la: `GlobalExceptionHandler`, `TestcontainersConfiguration`, `CategoryController.createCategory`, testele de categorii (`CategoryIntegrationTest`, `CategoryControllerTest`, `CategoryCommandServiceImplTest`), `SlotMateApiApplication` · **Repară: ELEV** (`createdAt` fără `@Builder.Default` rămânea null) · fără unealtă nouă: Spring Security + JWT cu `jjwt` și filtrul tău, ca la lecția de Spring Security

1. **Bază.** Tabela `users` din entitate (`ddl-auto: update`), cu coloanele din `STRUCTURA_BD.pdf`; migrarea `V2` o scrii în F5c.
2. **Model/repository.** `User` cu UUID și enum `Role` (`CLIENT`, `PROVIDER`, `ADMIN`, `AGENT`). `createdAt` se setează în `@PrePersist`, nu printr-un câmp de builder fără default. `UserRepository.findByEmail`.
3. **Permisiuni.** Enum-ul `Permission` complet (cele 13 din tabel) și o singură hartă `Role` → permisiuni (`RolePermissions`). La login, token-ul primește pe lângă `role` și claim-ul `permissions` (lista din `RolePermissions`). `JwtAuthFilter` (`OncePerRequestFilter`, înaintea `UsernamePasswordAuthenticationFilter`): citește `Authorization: Bearer …`, verifică semnătura și expirarea cu `jjwt` și `JWT_SECRET`, iar din claim-ul `permissions` face câte un `SimpleGrantedAuthority`, fără prefix; principalul e userId-ul din `sub`. Pe un token stricat, semnat cu altă cheie sau expirat, filtrul nu aruncă: doar nu autentifică, iar pe o rută protejată 401-ul îl dă `AuthenticationEntryPoint`-ul (excepțiile din filtru nu trec prin `GlobalExceptionHandler`). Harta există doar în API, la login. Fiecare șir din `hasAuthority(...)` există în enum.
4. **Service.** `AuthService.register`: doar `CLIENT` / `PROVIDER`, parola prin BCrypt. `login`: JWT HS256 făcut cu `jjwt` (`JwtService`), cu `sub` = userId, email, `role` și `permissions` = lista din `RolePermissions` pentru rol, `exp = iat + 3600`. Email inexistent și parolă greșită dau același 401. Un `ApplicationRunner` creează contul `ADMIN` din `ADMIN_EMAIL` / `ADMIN_PASSWORD`, dacă nu există.
5. **Intrare.** `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/users/me`. Rutele publice din tabelul de sus, inclusiv `/error` (fără el, orice excepție devine 401) și cele trei rute ale Swagger UI (`/swagger-ui.html`, `/swagger-ui/**`, `/v3/api-docs/**`), toate `permitAll`; restul cere token. `@EnableMethodSecurity`; `createCategory` cere `CATEGORY_MANAGE`. 401 și 403 ies ca ProblemDetail. springdoc declară `bearerAuth`; în `openapi.yaml`, la `bearerAuth`, descrierea primește și claim-ul `permissions`.
6. **Test cu login real.** `AuthTestHelper`: register → login → token, plus `adminToken()`: login real pe adminul creat de `ApplicationRunner` din `ADMIN_EMAIL` / `ADMIN_PASSWORD` puse în proprietățile de test. **De aici, toate testele pe endpoint-uri protejate îl folosesc**, nu `@WithMockUser`. Matricea pe POST categorii: anonim 401, `CLIENT` 403, `ADMIN` 201.
7. **Testele vechi, în același commit cu securitatea.** POST `/api/categories` devine protejat, deci testele de categorii ar primi 401. `CategoryIntegrationTest`: fiecare POST trimite tokenul de `ADMIN` din `adminToken()`. `CategoryControllerTest` e `@WebMvcTest`, fără bază, deci fără login real posibil: devine `@SpringBootTest` + Testcontainers, cu aceleași cazuri (201, 400 pe nume, 409) pe tokenul de `ADMIN`, iar mock-urile de service dispar. `CategoryCommandServiceImplTest` nu trece prin HTTP și rămâne cum e. `./gradlew test` e verde doar cu toate trei. `AuthService` are și teste Mockito: register cu email existent → 409, login cu parolă greșită → 401, parola salvată e hash-ul.
8. **Log.** Login reușit: `INFO` cu userId. Login eșuat: `WARN` cu email-ul. Parola nu apare niciodată, nici în log, nici în răspuns.
9. **Livrare.** Ca în F1: imagine nouă, `compose pull && up -d` pe EC2. `JWT_SECRET`, `ADMIN_EMAIL`, `ADMIN_PASSWORD` în `.env` pe EC2 (și în `.env.example`, fără valori). Testele își generează secretul lor.

Reguli

- Email valid și unic (409); parolă de minim 8 caractere; nume obligatoriu; telefon opțional (400 pe validare). Prin înregistrare nu se creează `ADMIN` sau `AGENT` (400).
- Login greșit → 401, același mesaj pentru email inexistent și parolă greșită. Token valabil o oră; în bază stă doar hash-ul parolei.
- Fără token pe rută protejată → 401; cu token fără permisiune → 403.

Gata când: pe EC2: register Ana (`CLIENT`) → 201, fără parolă în corp · register cu `"role":"ADMIN"` → 400 · login Ana → token cu `exp − iat = 3600` și `permissions = [BOOKING_CREATE, BOOKING_VIEW, BOOKING_CANCEL, ASSISTANT_USE, NOTIFICATION_VIEW]` · `/api/users/me` cu tokenul Anei → 200, `CLIENT` ; fără token → 401 · parolă greșită și email inexistent → 401, același detail · POST categorie cu tokenul Anei → 403, cu adminul din `.env` → 201 · `GET /api/categories` fără token → 200 · `/v3/api-docs` fără token → 200 · un token cu semnătura schimbată pe `/api/users/me` → 401 ProblemDetail · `./gradlew test` verde local, cu testele vechi de categorii pe tokenul de `ADMIN` .

F3 Firma mea și căutarea de furnizori

furnizor · vizitator

Furnizorul își creează profilul de firmă (nume, categorie, descriere, oraș, adresă), apoi și-l vede și îl modifică. Oricine caută furnizori după categorie, oraș sau amândouă și vede profilul unui furnizor.

Pornești de la: `Category` , `CategoryRepository` , `GlobalExceptionHandler` , `AuthTestHelper` , `User` , `RolePermissions` (F2) · **Repară: ELEV** (`@NotNull` pe relații, nu `@NotBlank` ; fără `cascade = ALL` pe category; coloana `address` , nu `adress`)

1. **Bază.** Tabela `providers` din entitate: `user_id` unic prin `uk_providers_user` în `@Table(uniqueConstraints = …)` și index `(category_id, city)` pentru căutare în `@Table(indexes = …)` . Migrarea `V3` vine în F5c.
2. **Model/repository.** `Provider` cu `@OneToOne` user și `@ManyToOne` LAZY category, fără cascade: altfel ștergerea unui furnizor ar șterge categoria tuturor. `existsByUserId` , `findByUserId` și o căutare cu filtrele opționale `categoryId` și `city` .
3. **Service.** Command: `createMyProvider` / `updateMyProvider` , proprietarul vine din token, nu din corp. Query: `getProvider` , `searchProviders` (ordonați după `businessName`), `getMyProvider` .
4. **Intrare.** Cele 5 operații din contract: `POST /api/providers` , `GET` / `PUT /api/providers/me` , `GET /api/providers` , `GET /api/providers/{providerId}` . `/api/providers/me/**` stă înaintea GET-urilor publice `/api/providers/**` .
5. **Permisiuni.** `PROVIDER_PROFILE_MANAGE` pe create/update; un `CLIENT` → 403.
6. **Test cu login real.** Trei conturi: furnizor A, furnizor B, client. Create 201, a doua oară 409, clientul 403, categorie inexistentă 404, căutarea pe cele 4 combinații de filtre.
7. **Log.** `INFO` la crearea profilului, cu providerId și userId.

Reguli

- Doar un cont `PROVIDER` are profil de firmă (403), și cel mult unul (409). Categoria trebuie să existe (404); id care nu e UUID → 400.
- `businessName` obligatoriu ≤ 150, `city` ≤ 100, `address` ≤ 255 (400).

Gata când: pe EC2: P1 (Frizerie, Cluj), P2 (Barber, Cluj), P3 (Frizerie, Iași) → 3 × 201 · al doilea POST al lui P1 → 409 · Ana → 403 · `?categoryId=<Frizerie>` → 2 · `?city=Cluj` → 2 · amândouă → 1 (P1) · fără filtre → 3 · `/api/providers/<UUID inexistent>` → 404 · `/api/providers/abc` → 400.

F4 Serviciile firmei

furnizor · vizitator

Furnizorul adaugă servicii (nume, descriere, durată în minute, preț), le modifică și le retrage. Oricine vede serviciile active ale unui furnizor.

Pornești de la: `Provider` , `ProviderRepository` , `ProviderQueryService` (F3), `AuthTestHelper` (F2) · **Repară: ELEV** (`Offering.price` era `Double` : banii devin `BigDecimal` / `DECIMAL(10,2)`)

1. **Bază.** Tabela `offerings` din entitate: preț pe `DECIMAL(10,2)` (`@Column(precision = 10, scale = 2)`), `active` implicit adevărat. Durata și prețul se verifică prin validare (400); `CHECK` -urile din bază vin cu `V4` , în F5c.
2. **Model/repository.** `Offering` cu `BigDecimal price` și `active` . `findByProviderIdAndActiveTrueOrderByName` .
3. **Service.** Furnizorul trebuie să aibă profil (404). **Proprietarul se verifică într-o singură metodă, `requireOwnOffering` ; F5, F7 și F10 o refolosesc.** `deactivateOffering` pune `active = false` ; rândul rămâne.
4. **Intrare.** `POST /api/offerings` , `PUT` și `DELETE /api/offerings/{offeringId}` , `GET /api/providers/{providerId}/offerings` (public).
5. **Permisiuni.** `OFFERING_MANAGE` + proprietarul: serviciul altui furnizor → 403.
6. **Test cu login real.** A modifică serviciul lui B → 403; durata 0 și prețul -1 → 400; după retragere, lista publică are cu unul mai puțin.
7. **Log.** `INFO` la create / deactivate, cu offeringId.

Reguli

- Furnizorul trebuie să aibă deja profil de firmă (404). Durata > 0; prețul ≥ 0 (400).
- Un furnizor modifică și retrage doar serviciile lui (403); retras = nu mai apare public, dar intervalele și rezervările lui rămân.

Gata când: pe EC2, cu tokenul lui P1: „Men's haircut” 30 min, 45.50 → 201, `price = 45.50` · „Beard trim” 20 min, 30.00 → 201 · `/api/providers/<P1>/offerings` fără token → 2 · `durationMinutes` 0 → 400; `price` -1 → 400 · PUT cu tokenul lui P3 → 403 · DELETE „Beard trim” → 204 → lista publică are 1 element, iar în bază rândul există cu `active = 0` .

F5 Intervalele unui serviciu

furnizor · vizitator

Furnizorul publică un interval pentru un serviciu dând doar ora de început; sfârșitul se calculează din durată. Oricine vede intervalele dintre două date (toate sau doar cele libere), ordonate după oră. Furnizorul șterge un interval nerezervat.

Pornești de la: `Offering` , `OfferingRepository` , `requireOwnOffering` (F4), `AuthTestHelper` (F2)

1. **Bază.** Tabela `slots` din entitate, cu `uk_slots_offering_start` declarată în `@Table(uniqueConstraints)` . Migrarea `V5` vine în F5c.
2. **Model/repository.** `Slot` (offering, startTime, endTime). Căutare după serviciu între două momente, ordonată după oră, și `existsByOfferingIdAndStartTime` .
3. **O singură cale de publicare.** `publishSlot(offering, startTime)` : viitorul, `endTime = start + durată` , duplicatul (409, prins și din `DataIntegrityViolationException`). `createSlot` = proprietarul + `publishSlot` . **F7 și F10 o refolosesc; nu se scrie alta.** `deleteSlot` : proprietarul + interval nerezervat. `booked` e false până la F6.
4. **Intrare.** `POST /api/offerings/{offeringId}/slots` , `GET /api/offerings/{offeringId}/slots?from&to&available` (public), `DELETE /api/slots/{slotId}` . Timpul e `LocalDateTime` fără fus, cum spune contractul.
5. **Permisiuni.** `SLOT_MANAGE` + proprietarul serviciului (403).
6. **Test cu login real.** Ordinea la GET, 409 la duplicat, 400 în trecut, 403 la alt furnizor.
7. **Log.** `INFO` la publicare și ștergere, cu slotId și offeringId.

Reguli

- Ora de început e în viitor (400); `endTime = startTime + durationMinutes` . Aceeași oră de început la același serviciu → 409.
- Un interval rezervat nu se șterge (409, activ de la F6); un furnizor publică și șterge doar la serviciile lui (403).
- GET: `from` și `to` sunt date (inclusiv), obligatorii (400 dacă lipsesc); ordonat după `startTime` .

Gata când: pe EC2, cu tokenul lui P1, pe „Men's haircut”, o zi viitoare D care nu e marți (marțea e a programului din F7): POST 10:00 → 201, `endTime` = D 10:30, `booked = false` · același POST → 409 · o oră de ieri → 400 · POST 09:00 și 11:00 → GET `?from=D&to=D` fără token → 3, în ordinea 09:00, 10:00, 11:00 · DELETE 11:00 cu tokenul lui P3 → 403; cu al lui P1 → 204 → GET = 2.

F5b Livrarea pleacă singură la push

(dezvoltatorul)

Ce ai făcut de mână în F1–F5 (test, imagine, scp, ssh) se face singur la fiecare push pe master. Fără cod de domeniu nou: aceleași endpoint-uri, livrate de pipeline.

Pornești de la: `slot-mate-api/Dockerfile`, `slot-mate-ops/docker-compose.yml`, testele din F1–F5 · **nou** GitHub Actions + GHCR, cu profilurile compose

1. **Workflow.** `.github/workflows/api.yml`, pe push pe master, `runs-on: ubuntu-latest` (are Docker, deci Testcontainers merge), cu trei joburi la rând.
2. **`test`.** `actions/checkout`, `actions/setup-java` pe `temurin` 25, apoi testele Gradle.
3. **`image`.** `permissions: packages: write`; login pe `ghcr.io` cu `GITHUB_TOKEN`; build + push `ghcr.io/tomescuflavius/slot-mate-api` cu tag-urile `latest` + SHA, pentru arhitectura instanței EC2. În compose, imaginea de pe Docker Hub devine `ghcr.io/tomescuflavius/slot-mate-api:${API_TAG:-latest}`.
4. **Profiluri compose.** Fiecare serviciu din compose primește un profil: mysql și api = `core`. Pe EC2, `.env` are `COMPOSE_PROFILES=core`, deci `up -d` pornește doar ce e în profilurile listate. Fiecare feature care adaugă un serviciu îi spune profilul; pe EC2 ajung doar `core`, `edge` (F6), `obs` (F8) și `auth` (F16). `messaging`, `logs` și `tracing` (partea III) nu încap: rulează doar local și pe instanța de demo, unde `COMPOSE_PROFILES` le are pe toate.
5. **`deploy`.** Secretele `EC2_HOST`, `EC2_USER`, `EC2_SSH_KEY`; cheia din secret într-un fișier cu drepturi 600, `scp` pe compose, apoi `ssh` + `API_TAG=${{ github.sha }} docker compose pull api && API_TAG=${{ github.sha }} docker compose up -d`, pașii tăi din F1 cu tag-ul commit-ului. `.env` rămâne pus de mână pe EC2, cu `COMPOSE_PROFILES`: deploy-ul automat pornește doar profilurile de acolo, niciodată partea III.

Reguli

- `image` rulează doar dacă `test` e verde, iar `deploy` doar dacă `image` e verde. Niciun secret în workflow sau în repo.

Gata când: un push pe master trece cele 3 joburi pe verde · pe EC2, `.env` are `COMPOSE_PROFILES=core`, iar `docker compose ps` arată doar mysql și api, cu imaginea nouă de pe `ghcr.io` (tag-ul = SHA-ul commit-ului) · `GET http://<ip>:8080/api/categories` → 200, tot cele 3 categorii din F1 · furnizorii, serviciile și intervalele din F3–F5 sunt tot acolo.

F5c Schema bazei intră în Git

(dezvoltatorul)

Până acum Hibernate a creat tabelele singur. De aici, schema e scrisă în fișiere SQL versionate, aceleași pe laptop, în teste, în Actions și pe EC2. Fără cod de domeniu nou.

Pornești de la: entitățile din F1–F5 (`Category`, `User`, `Provider`, `Offering`, `Slot`), tabelele create de Hibernate în baza locală, `application.yaml` · **nou** Flyway

1. **Bază.** `spring-boot-starter-flyway` + `flyway-mysql`. Câte o migrare pe feature, în `src/main/resources/db/migration`: `V1__categories.sql`, `V2__users.sql`, `V3__providers.sql`, `V4__offerings.sql`, `V5__slots.sql`, **fără rânduri de seed**. Pornești de la ce a creat Hibernate (`mysqldump --no-data` pe baza locală) și compari cu `STRUCTURA_BD.pdf`: `user_id` unic și indexul `(category_id, city)` pe `providers`, prețul pe `DECIMAL(10,2)` și `CHECK`-urile pe durată și preț la `offerings`, `uk_slots_offering_start` pe `slots`.
2. **Config.** `ddl-auto: validate`: de acum Hibernate doar verifică, nu mai creează nimic. Flyway rulează și în teste, deci `validate` dovedește că migrările = entitățile.
3. **Baza pornește curat, o singură dată.** Flyway nu rulează V1 peste tabele care există deja. Local ștergi `slot_mate_api_db`; pe EC2, `docker compose down -v` (șterge volumul MySQL), apoi `up -d`: Flyway rulează V1–V5 pe baza goală, iar adminul revine din `.env`. Categoriile, furnizorii, serviciile și intervalele de test le pui din nou, prin Swagger.
4. **Test.** Testele din F1–F5 rămân neschimbate și trec în Actions: tabelele vin acum din migrări, nu din entități.

Reguli

- O migrare deja rulată nu se mai modifică niciodată; o schimbare = un `Vn` nou. F6 continuă cu `V6__bookings.sql`.

Gata când: local, pe bază ștearsă: aplicația pornește, `flyway_schema_history` are 5 rânduri (V1–V5, success = 1) · push pe master → cele 3 joburi din F5b verzi · pe EC2, după `down -v`: `flyway_schema_history` are aceleași 5 rânduri · `SHOW CREATE TABLE offerings` are `CHECK`-urile pe durată și preț · „Gata când" din F1–F5, refăcut prin Swagger, trece la fel.

F6 Rezerv un interval

client · furnizor

Clientul rezervă un interval liber (cu o notă opțională), își vede rezervările și le poate anula. Furnizorul vede rezervările de la firma lui pe o perioadă și le poate anula; un interval anulat redevine liber.

Pornești de la: `Slot`, `SlotRepository`, `publishSlot`, `deleteSlot` (F5), `AuthTestHelper` (F2), `slot-mate-ops/docker-compose.yml` (F1) · **nou** nginx + certbot (HTTPS pe domeniu)

1. **Bază.** `V6__bookings.sql` cu `uk_bookings_slot` pe `slot_id`.
2. **Model/repository.** `Booking` (slot, client, note, createdAt). Rezervările unui client ordonate după ora intervalului; ale unui furnizor într-un interval de timp. De acum `booked` vine din bază, **cu o singură interogare pentru toată lista, nu N**.
3. **Service.** `createBooking`: doar `CLIENT`, interval în viitor, interval deja rezervat → 409, prins și din `DataIntegrityViolationException`. `cancelBooking` șterge rândul: clientul rezervării sau furnizorul intervalului. `deleteSlot` pe interval rezervat → 409.
4. **Intrare.** `POST /api/bookings`, `GET /api/bookings/me`, `GET /api/providers/me/bookings?from&to`, `DELETE /api/bookings/{bookingId}`.
5. **Permisiuni.** `BOOKING_CREATE`, `BOOKING_VIEW`, `BOOKING_CANCEL`, cu proprietarul verificat în service.
6. **Test cu login real.** Rezervă 201 → a doua oară 409 → anulează 204 → rezervă din nou 201. Concurență: 10 clienți reali, 10 thread-uri pe același interval → exact un 201.
7. **Log.** `INFO` la rezervare și anulare, cu bookingId, slotId și cine a anulat.
8. **Livrare pe HTTPS.** De acum clienții își trimit pe internet parola, tokenul și rezervările: nu mai merg în clar. Un domeniu (sau subdomeniu) `api.<domeniu>` cu A record pe Elastic IP-ul EC2. nginx (profilul `edge`) termină TLS cu certificat Let's Encrypt, reînnoit de containerul certbot, și trimite tot la `api:8080`, fără logică. `nginx.conf` în `slot-mate-ops/nginx/`. Al doilea nume, `app.<domeniu>`, pe același certificat, pentru UI-ul din U1: `/` → `ui:80`, iar `/api/` → exact rutele lui `api.<domeniu>`, scrise o singură dată în `api-locations.conf` și incluse în ambele server block-uri (F12, F14 și F17 schimbă doar fișierul ăsta). ui nu mai publică portul 80, iar nginx-ul din imaginea lui rămâne doar cu fișierele statice. api nu mai publică portul 8080; Security Group: 80 și 443 deschise, 22 doar de la IP-ul tău. `COMPOSE_PROFILES=core,edge`. Swagger UI de pe EC2 are 21 de operații, cât `openapi.yaml`.

Reguli

- Doar un cont `CLIENT` rezervă (403); intervalul e în viitor (400); slot inexistent → 404.
- Un interval are cel mult o rezervare (409), inclusiv când doi clienți îl cer în aceeași secundă.
- Anulează doar clientul rezervării sau furnizorul intervalului (403); anularea → 204, iar intervalul redevine liber. Rezervările clientului, ordonate după ora intervalului.

Gata când: pe EC2, prin `https://api.<domeniu>`, pornind de la ziua D lăsată de F5 (09:00 și 10:00; 11:00 a fost șters): P1 republică 11:00 → 201 → GET = 3 (dacă D a trecut, P1 publică pe o nouă zi D, tot nu marți, 09:00, 10:00 și 11:00 → tot 3) · Ana → 10:00 → 201; din nou → 409 · `available=true` → 2 (09:00, 11:00) · tokenul lui P1 pe POST rezervare → 403 · Ana anulează → 204 → 3 libere · Ana rezervă din nou 10:00 → 201 · 10 clienți reali în paralel pe 11:00 → 1 × 201 și 9 × 409 → `available=true` → 1 (09:00) · `/api/providers/me/bookings?from=D&to=D` cu P1 → 2 (10:00, 11:00) · un alt client anulează rezervarea Anei → 403 · Swagger UI → 21 de operații · `curl -m 5 http://<ip>:8080/api/categories` din afară → timeout; certificatul de pe `api.<domeniu>` e valid · `https://app.<domeniu>` → UI-ul din U1, iar Ana se loghează în el ca înainte; în `docker compose ps`, ui nu mai are port publicat.

F7 Programul săptămânal, publicat singur

furnizor · (job-ul de noapte)

Furnizorul își declară o singură dată programul pe serviciu (marți, 09:00–12:00), iar în fiecare noapte aplicația publică singură intervalele pentru următoarele 14 zile, fără să dubleze ce există. Fără el, un frizer ar crea de mână zeci de intervale pe săptămână. Primul cod care rulează fără request HTTP.

Pornești de la: `SlotMateApiApplication`, `application.yaml`, `publishSlot`, `deleteSlot`, `Slot`, `SlotRepository` (F5), `requireOwnOffering` (F4) · **nou** logare structurată (MDC cu runId, JSON pe EC2)

1. **Bază.** `V7__weekly_schedules.sql` + `slots.source` (`MANUAL` / `SCHEDULE`) + `slot_exclusions`.
2. **Model/repository.** `WeeklySchedule` (offering, `Integer dayOfWeek` 1–7, nu `DayOfWeek` cu ORDINAL, startTime, endTime); programele serviciilor active. `Slot` primește `source`. `SlotExclusion` (offering, startTime) și repository-ul ei.
3. **Service.** `putSchedule` înlocuiește tot programul serviciului. Un bean `Clock` (Europe/Bucharest), injectat peste tot unde se citește „acum”, ca testele să fixeze timpul. `SlotGenerationJob`: pentru D+1..D+14 și fiecare serviciu activ cu program, cheamă `publishSlot(…, SCHEDULE)` din F5. Ce există se sare, la fel și excluderile; UQ-ul rămâne plasa de siguranță. `deleteSlot` (F5) pe un interval care cade în programul serviciului scrie o excludere (offering, startTime); `publishSlot` pe aceeași oră (manual sau `ADD` aprobat) șterge excluderea.
4. **Intrare.** `PUT` / `GET /api/offerings/{offeringId}/schedule` (contractul trece la 1.3.0). Job-ul: cron din `SLOT_GENERATION_CRON` (implicit 01:00, Europe/Bucharest), plus o rulare la pornire când `SLOT_GENERATION_ON_STARTUP=true`.
5. **Permisiuni.** `SLOT_MANAGE` + proprietarul pe PUT; GET-ul programului e tot al proprietarului (403 pentru alții).
6. **Test cu login real.** `putSchedule` ca P1 și ca P3 (403). Job-ul cu `Clock.fixed`: număr exact de intervale; a doua rulare nu creează nimic.
7. **Logare.** Fiecare rulare primește un `runId` în MDC; `offeringId` intră în MDC pe serviciul curent. O linie `INFO` per rulare („slot generation finished”, cu offerings, created, skipped, durationMs), `DEBUG` per serviciu. Un filtru HTTP pune `requestId` și `userId` în MDC. Pe EC2 logurile ies JSON (`ecs`); local, text.
8. **Livrare.** `SPRING_PROFILES_ACTIVE=prod` în `.env` pe EC2.

Reguli

- `dayOfWeek` 1–7 (luni = 1); ferestrele aceluiași serviciu nu se suprapun (409); start < end și fereastra e multiplu de durata serviciului, altfel 400.
- Job-ul publică doar în viitor, pe D+1..D+14, și nu atinge intervalele existente: `MANUAL` rămâne `MANUAL`. Serviciile retrase nu primesc intervale noi.
- Două rulări la rând dau același rezultat (a doua: `created = 0`). Un interval șters nu revine la rularea următoare.

Gata când: pe EC2, P1 pe „Men's haircut”: PUT marți 09:00–12:00 → 200; 09:00–12:10 → 400; tokenul lui P3 → 403 · un interval `MANUAL` marțea viitoare la 10:00 · repornit cu `SLOT_GENERATION_ON_STARTUP=true`: 2 marți × 6 = 12 așteptate, 1 există → `created = 11`, `skipped = 1`; GET pe fiecare din cele două marți → 6 + 6 = 12 (11 `SCHEDULE`; ziua D din F5–F6 nu e marți, deci nu se amestecă) · a doua rulare → `created = 0`, `skipped = 12` · DELETE un interval `SCHEDULE` → încă o rulare: `created = 0`, intervalul tot lipsă · `docker compose logs api | grep <runId>` → doar liniile acelei rulări, JSON cu `runId`.

F8 Ocuparea, văzută live

furnizor · administrator

Furnizorul își vede ocuparea pe zile și ore din ultimele săptămâni; administratorul o vede pe toți. În Grafana se văd live ocuparea pe categorii, rezervările, anulările și conflictele: ocupare mică = furnizori care pleacă, multe conflicte = prea puține intervale.

Pornești de la: `Booking`, `createBooking`, `cancelBooking` (F6), `SlotGenerationJob` (F7), `slot-mate-ops/docker-compose.yml` (F1) · **nou** Micrometer + Prometheus + Grafana

1. **Bază.** Fără migrare: ocuparea se calculează din `slots` + `bookings`, pe indexul existent `(offering_id, start_time)`.
2. **Model/repository.** O agregare pe serviciu × (zi, oră): intervale publicate și rezervate, cu `start_time` în fereastra **[azi 00:00 – 7 × `weeks` zile, azi 00:00)**, unde „azi” e data din bean-ul `Clock` din F7 (Europe/Bucharest), același fus în care stau intervalele (`LocalDateTime`). Ziua de azi nu intră; capătul din stânga intră, cel din dreapta nu. Cu `weeks = 4` fereastra are exact 28 de zile, deci exact 4 zile din fiecare zi a săptămânii.
3. **Service.** `OccupancyQueryService.getOccupancy(providerId, weeks)` → bucket-uri {offeringId, dayOfWeek, startTime, published, booked, rate} + totalurile. Proprietarul se verifică într-un singur loc: `PROVIDER` doar firma lui, `ADMIN` / `AGENT` toate. F9 și F11 o refolosesc.
4. **Intrare.** `GET /api/providers/me/occupancy?weeks=4` și `GET /api/providers/{providerId}/occupancy?weeks=4` (contractul trece la 1.4.0).
5. **Permisiuni.** `STATS_VIEW`, cu firma verificată în service (alt furnizor → 403).
6. **Test cu login real.** Ocupare pe date puse prin `Clock` fix; P2 pe P1 → 403; `ADMIN` pe P1 → 200.
7. **Metrici.** Actuator + `micrometer-registry-prometheus`. Contoare: `slotmate.bookings.created`, `slotmate.bookings.cancelled{by}`, `slotmate.bookings.conflicts` (fiecare 409 pe interval ocupat), `slotmate.slots.generated{result}`. Timer: durata job-ului din F7. Gauge: ocuparea intervalelor din următoarele 7 zile (azi exclus), pe categorie. `/actuator/prometheus` pe management port 8081, nepublicat.
8. **Permisiuni pe actuator.** `/actuator/health` și `/actuator/prometheus` sunt `permitAll` în `SecurityFilterChain` (Prometheus face scrape fără token); le protejează faptul că portul 8081 nu e publicat în compose și nici în Security Group, nu tokenul.
9. **Ops.** Prometheus și Grafana în compose, profilul `obs` (`COMPOSE_PROFILES=core,edge,obs` pe EC2); dashboard-ul (JSON) și sursa de date în `slot-mate-ops/grafana/`, încărcate automat. Grafana pe EC2 doar de la IP-ul tău (Security Group), parola de admin în `.env`. `DemoDataSeeder` (`@Profile("demo")`) nu face nimic dacă există furnizori; altfel scrie prin repository-uri 3 categorii, 6 furnizori cu program și intervalele programului cu start în **[ziua seed-ului 00:00 – 28 de zile, ziua seed-ului 00:00)**, pe aceeași oră a `Clock`-ului, cu rezervări după o regulă fixă. La P1 (4 marți în fereastră, oricare ar fi ziua seed-ului): 09:00 niciuna, 09:30 și 10:30 în toate marțile în afară de cea mai veche, 10:00, 11:00 și 11:30 în toate 4. Conturile au nume fixe și parola din `DEMO_PASSWORD` (în `.env`): P1 = `p1@demo` (Frizerie, Cluj, „Men's haircut” 30 min, program marți 09:00–12:00), P2 = `p2@demo` (Barber, Cluj), Ana = `ana@demo` (`CLIENT`), `client01` … `client20`.
10. **Livrare.** Pe EC2, o singură dată: `docker compose down -v` (șterge volumul MySQL; Flyway reface V1–V7 la pornire, adminul revine din `.env`), apoi `SPRING_PROFILES_ACTIVE=prod,demo` și `DEMO_PASSWORD` în `.env`.

Reguli

- Ocupare = rezervate / publicate, doar în fereastra de mai sus (azi și viitorul nu intră); bucket fără intervale publicate nu apare (nu se împarte la 0). Cifrele depind de ziua în care întrebi: verificarea se face în aceeași zi cu seed-ul.
- `weeks` între 1 și 8 (400 în afară). Endpoint-ul de metrici nu se vede din internet.

Gata când: pe EC2, profilul demo pe bază goală, verificat în aceeași zi cu seed-ul: în fereastra de 28 de zile, P1, „Men's haircut” = 4 marți × 6 = 24 publicate; rezervate 09:00 0/4, 09:30 3/4, 10:00 4/4, 10:30 3/4, 11:00 4/4, 11:30 4/4 → 0 + 3 + 4 + 3 + 4 + 4 = 18 · `/api/providers/me/occupancy?weeks=4` cu P1 → `published = 24`, `booked = 18`, `rate = 0.75` · `/api/providers/<P1>/occupancy` cu P2 → 403, cu adminul → 200 · 20 de clienți demo în paralel pe un interval liber de marțea viitoare (publicat de job) → 1 × 201 și 19 × 409, iar panoul de conflicte crește cu 19 · `curl -m 5 http://<ip>:8081/actuator/prometheus` din afară → timeout (portul nu e în Security Group); pe EC2, `docker compose exec prometheus wget -qO- http://api:8081/actuator/prometheus | grep slotmate_bookings` → răspunde (scrape-ul din rețeaua compose merge).

II. MCP — piesa centrală

Platforma devine ceva ce poate folosi un agent: întâi citind, apoi propunând. Apar primul microserviciu și asistentul care le găsește clienților un loc.

F9 Platforma văzută de un agent

orice client MCP cu token

Un client MCP (MCP Inspector, mai târziu agentul tău) se conectează cu un token și întreabă de categorii, furnizori, servicii, intervale libere, rezervări și ocupare. Vede exact cât vede utilizatorul tokenului.

Pornești de la: query service-urile din F1–F8 (`getAllCategories`, `ProviderQueryService`, `OfferingQueryService`, `SlotQueryService`, `BookingQueryService`, `OccupancyQueryService`), `SecurityFilterChain` (F2), `build.gradle` · **nou** server MCP (Spring AI 2.0.2, `spring-ai-starter-mcp-server-webmvc`)

1. **Dependență.** Plugin-ul Boot trece de la 4.1.1 la 4.1.2 (patch, cerut de starter), plus `spring-ai-bom` 2.0.2 și starter-ul de server. Transport Streamable HTTP pe `/mcp`. Versiunile le verifici pe Maven Central în ziua în care le adaugi.
2. **Bază / model.** Nimic nou.
3. **Tool-uri, în pachetul `mcp/`.** `listCategories`, `searchProviders(categoryId?, city?)`, `getProvider`, `getProviderOfferings`, `getSlots(offeringId, from, to, available)`, `getMyBookings`, `getMyProviderBookings(from, to)`, `getOccupancy(providerId, weeks ≤ 8)`.
4. **Regula tool-urilor.** Un tool = un apel la un service existent: fără repository, fără logică în tool. Descrierea tool-ului și a fiecărui parametru, în engleză, e scrisă pentru un model.
5. **Permisiuni.** `/mcp` stă în spatele aceluiași filtru JWT (fără token → 401). Proprietarul se verifică în service, deci REST și MCP văd la fel.
6. **Test cu login real.** Cereri JSON-RPC pe `/mcp` cu `RestTestClient` și tokenul din login: `initialize` → `tools/list` (8 tool-uri) → `tools/call getSlots`, același rezultat ca REST; datele altcuiva → eroare de tool. MCP Inspector pentru verificarea de mână.
7. **Log / metrică.** Fiecare apel: o linie `INFO` cu tool-ul și userId, plus contorul `slotmate.mcp.calls{tool}`, vizibil în Grafana.

Reguli

- Tool-urile de citire nu schimbă nimic în bază. Fără token → 401 înainte de orice tool.
- Erorile de acces și de validare ajung la clientul MCP ca erori de tool, cu același mesaj ca ProblemDetail-ul REST.

Gata când: pe EC2, cu datele demo din F8 puse în aceeași zi (`docker compose down -v`, profilul demo pornit pe bază goală), MCP Inspector pe `https://api.<domeniu>/mcp` cu tokenul Anei → 8 tool-uri · `getSlots` („Men's haircut", marțea viitoare, libere) = același număr ca REST · Ana pe `getMyProviderBookings` → eroare de acces · P2 pe `getOccupancy(P1)` → eroare de acces; P1 → `published = 24`, `booked = 18` · fără token → 401.

F10 Agentul propune, furnizorul aprobă

agent · furnizor

Un agent propune ajustări de program pentru firma unui furnizor: un interval în plus după o zi plină (`ADD`) sau scoaterea unui interval liber dintr-o oră în care nu rezervă nimeni (`REMOVE`). Propunerea așteaptă; doar furnizorul o aprobă, iar execuția trece prin aceeași publicare / ștergere ca acțiunea lui manuală.

Pornești de la: `publishSlot`, `deleteSlot` (F5), `requireOwnOffering` (F4), `ApplicationRunner`-ul de admin (F2), serverul MCP (F9) · **nou** tool-uri MCP de scriere: propunere → aprobare de un om → aceeași cale de execuție

1. **Bază.** `V8__slot_proposals.sql` (`proposal_runs` + `slot_proposals`). Contul `AGENT` îl creează același `ApplicationRunner`, din `AGENT_EMAIL` / `AGENT_PASSWORD`, dacă nu există.
2. **Model/repository.** `ProposalRun`, `SlotProposal` și repository-urile lor.
3. **Service.** `proposeSlotPlan(providerId, items)` salvează run-ul și fiecare propunere, trecută prin aceleași validări ca `publishSlot` / `deleteSlot`, **fără să atingă intervalele**: `valid` + `rejection_reason` (`SLOT_IN_PAST`, `SLOT_ALREADY_EXISTS`, `SLOT_BOOKED`, `SLOT_NOT_FOUND`, `OFFERING_INACTIVE`, `NOT_PROVIDER_OFFERING`). `approve`: `ADD` → `publishSlot(…, AGENT)`, `REMOVE` → aceeași ștergere ca `deleteSlot`, deci și excluderea din F7; rezervat între timp → `FAILED` cu motivul. `reject` → `REJECTED`.
4. **Intrare.** Tool-uri MCP `proposeSlotPlan` (`SLOT_PROPOSE`) și `getProposalRun(runId)` (`SLOT_PROPOSAL_VIEW`). REST: `GET /api/providers/me/slot-proposals`, `POST /api/slot-proposals/{proposalId}/approve` și `/reject` (`SLOT_PROPOSAL_APPROVE`). Contractul trece la 1.5.0.
5. **Permisiuni.** `AGENT` are `SLOT_PROPOSE`, dar **nu** `SLOT_MANAGE` și nici `SLOT_PROPOSAL_APPROVE`: nu publică direct și nu se aprobă singur.
6. **Test cu login real.** `AGENT` propune, P1 aprobă. Agentul pe aprobare → 403; P2 pe propunerile lui P1 → 403; agentul pe POST intervale → 403.
7. **Log / metrică.** `INFO` per run (runId, providerId, câte valide, câte respinse); contoarele `slotmate.proposals{status}`.
8. **Livrare.** `AGENT_EMAIL` și `AGENT_PASSWORD` în `.env` pe EC2.

Reguli

- Agentul nu execută direct: propune; execuția = `publishSlot` / `deleteSlot`, aceeași metodă și aceeași validare ca la furnizor. Cel mult 20 de propuneri într-un run (400).
- Propunere invalidă la propunere → nu se aprobă (409); una validă care devine invalidă până la aprobare → `FAILED`, fără să strice restul. Propunere deja decisă sau cu ora trecută → 409.
- Doar furnizorul firmei aprobă sau respinge.

Gata când: pe EC2, P1 cu marțea viitoare 09:00–11:30 publicată de job, Ana a rezervat 09:00 · din MCP Inspector, ca `AGENT`: `ADD` 12:00, `ADD` 10:00, `REMOVE` 09:00 → 1 validă, 2 invalide (`SLOT_ALREADY_EXISTS`, `SLOT_BOOKED`) · P1 aprobă → `EXECUTED` · GET marțea viitoare → 7 intervale (09:00…12:00), cel de la 12:00 cu `source = AGENT` · celelalte rămân neexecutate, cu motivul · `AGENT` pe `/approve` → 403 · P2 → 403 · `AGENT` pe POST intervale → 403.

F11 Planificatorul pe reguli — primul microserviciu

agent · furnizor (aprobă)

O aplicație separată, `slot-mate-agent`, citește noaptea prin MCP ocuparea fiecărui furnizor din ultimele 4 săptămâni, aplică regulile și propune ajustările pentru săptămâna viitoare. Nu are bază de date și nu cunoaște API-ul REST (în afară de login), doar tool-urile.

Pornești de la: tool-urile de citire din F9 (`searchProviders`, `getProviderOfferings`, `getSlots`, `getOccupancy`), `proposeSlotPlan` și contul `AGENT` (F10) · **nou** client MCP, a doua aplicație (`spring-ai-starter-mcp-client` 2.0.2)

1. **Proiect.** Folderul `slot-mate-agent/` în același repo: Boot 4.1.2, Java 25, Gradle. La pornire se loghează ca `AGENT` prin `POST /api/auth/login` (credențiale din `.env`) și trimite tokenul la `/mcp`; îl reînnoiește înainte să expire.
2. **Bază.** Niciuna.
3. **Calcul pur.** Interfața `SlotPlanner` și `RuleBasedPlanner`: din ocupare + intervalele săptămânii viitoare → propuneri. O metodă per regulă, cu teste unitare (în lucrare, fiecare devine un paragraf).
4. **Bucla.** La `AGENT_CRON` (implicit 02:00) și la pornire dacă `AGENT_RUN_ON_STARTUP=true`: `searchProviders` → per furnizor `getOccupancy(4)` + `getSlots` pe săptămâna viitoare → planner → `proposeSlotPlan`, doar dacă lista nu e goală. Fiecare rulare are un `runId` în MDC.
5. **Permisiuni.** Tokenul `AGENT` (`STATS_VIEW`, `SLOT_PROPOSE`, `SLOT_PROPOSAL_VIEW`). Că nu poate aproba s-a verificat în F10.
6. **Test.** Testele unitare ale regulilor + un test de integrare care pornește API-ul în Testcontainers, se loghează real ca `AGENT` și verifică run-ul creat.
7. **Log / metrică.** `INFO` per rulare (runId, furnizori, propuneri, durată); contorul `agent.runs` și un timer pe rulare, în același Prometheus.
8. **Livrare.** Dockerfile propriu, serviciu nou în compose (profilul `core`: rulează și pe EC2), job nou în Actions filtrat pe `slot-mate-agent/**`, imagine `ghcr.io/tomescuflavius/slot-mate-agent`. Agentul cheamă `http://api:8080/mcp` din rețeaua compose, nu prin internet.

Reguli

- **R1:** bucket (serviciu, zi, oră) cu ≥ 3 rezervări din 4 publicate, care e ultimul interval al zilei → `ADD` un interval imediat după el, în aceeași zi a săptămânii viitoare, dacă se termină până la 20:00 și nu există deja.
- **R2:** bucket cu 0 rezervări din 4 publicate → `REMOVE` intervalul liber din aceeași zi și oră a săptămânii viitoare (dacă e rezervat, nu se propune).
- Bucket-urile cu mai puțin de 4 intervale publicate se ignoră. Cel mult 5 propuneri per furnizor per rulare. Agentul oprit nu oprește platforma.

Gata când: pe EC2, cu datele demo din F8 puse în aceeași zi (`down -v` , profilul demo pe bază goală, job-ul din F7 a publicat marțea viitoare: 6 intervale, 09:00…11:30), agentul repornit cu `AGENT_RUN_ON_STARTUP=true` · `/api/providers/me/slot-proposals` cu P1 → un run cu exact 2: `ADD` marți 12:00 (bucket 11:30 = 4/4, ultimul, se termină la 12:30) și `REMOVE` marți 09:00 (0/4); la 10:00 și 11:00 (4/4, dar nu ultimele) nimic · P1 aprobă ambele → marțea viitoare 6 – 1 + 1 = 6 intervale, 09:30…12:00 · după rularea de noapte a job-ului, marțea viitoare tot 6 · `docker compose stop agent` → `GET /api/categories` tot 200 · în logurile agentului filtrezi rularea după `runId` .

F12 Asistentul care îți găsește un loc

client

Clientul scrie ce vrea în cuvintele lui („un tuns marțea viitoare dimineața, în Cluj”), iar asistentul îi propune cel mult 3 intervale libere, fiecare cu motivul ales. Clientul rezervă singur unul dintre ele. Fără model AI, asistentul caută după câmpurile completate.

Pornești de la: `slot-mate-agent` și clientul MCP (F11), tool-urile de citire (F9), `createBooking` (F6) · **nou** Spring AI `ChatClient` (cu tool-urile MCP și ieșire structurată)

1. **Bază.** Niciuna: sugestiile nu se salvează.
2. **Model.** `AssistantRequest` (message, categoryId?, city?, from?, to?) și `BookingSuggestion` (slotId, offeringId, providerName, offeringName, startTime, price, reason).
3. **Service.** `SuggestionService` , cu două implementări. `LlmSuggester` : `ChatClient` cu `listCategories` , `searchProviders` , `getProviderOfferings` , `getSlots` (modelul își cere singur datele) și ieșire structurată; prompt-ul, în engleză, stă într-un fișier din `resources` . `RuleBasedSuggester` : cele mai devreme 3 intervale libere după `categoryId` , `city` și from–to. **Orice sugestie a modelului se verifică din nou prin `getSlots(available = true)`** ; cele invalide se scot și se numără.
4. **Siguranță.** Timeout 20 s. Model care nu răspunde, răspunde greșit sau nu are nicio sugestie validă → `RuleBasedSuggester` + `WARN` . Modelul e Claude, prin API-ul Anthropic: `spring-ai-starter-model-anthropic` (Spring AI 2.0.2, verificat pe Maven Central), `spring.ai.anthropic.api-key=${ANTHROPIC_API_KEY}` , `spring.ai.anthropic.chat.options.model=${ANTHROPIC_MODEL:claude-opus-5-5}` . Cheia stă în `.env` local și ca secret în Actions pentru EC2, niciodată în repo; asistentul merge și pe EC2. Datele trimise modelului sunt doar ce întorc tool-urile MCP.
5. **Intrare.** `POST /api/assistant/suggestions` în `slot-mate-agent` , documentat în `slot-mate-docs/openapi-agent.yaml` ; nginx trimite `/api/assistant/` la agent, restul `/api` la api. Tokenul clientului merge mai departe la `/mcp` , deci asistentul vede exact cât vede clientul.
6. **Permisiuni.** `slot-mate-agent` validează același JWT (același `JWT_SECRET` și o copie a `JwtAuthFilter` -ului din F2 pe claim-ul `permissions` , fără o copie a hărții) și cere `ASSISTANT_USE` . Rezervarea e `createBooking` din F6, făcută de client.
7. **Test cu login real.** `RuleBasedSuggester` și verificarea sugestiilor, cu un model fals care întoarce un slotId ocupat (trebuie scos). Tokenul unui `PROVIDER` → 403.
8. **Metrici + comparația pentru lucrare.** Tokenii, latența, rata de fallback și sugestiile scoase, în Prometheus. 20 de cereri fixe (un fișier în `test/resources`) pe aceleași date demo, cu fiecare suggester: % sugestii valide, % în filtrele cerute, latență p50, tokeni.

Reguli

- Cel mult 3 sugestii, doar intervale libere și în viitor, ordonate după `startTime` . Asistentul nu rezervă și nu scrie nimic în bază.
- Fără model sau la timeout → fallback pe reguli, cu `WARN` ; fără câmpuri structurate și fără model → 400 („categoryId or city is required when no model is available”).

Gata când: local, cu modelul pornit, tokenul Anei: „A haircut next Tuesday morning in Cluj” → 1–3 sugestii libere, la o Frizerie din Cluj, înainte de 12:00 · Ana rezervă prima → 201 · aceeași cerere din nou → intervalul rezervat nu mai apare · pe EC2, fără model, cu `categoryId` = Frizerie, `city` = Cluj, from = to = marțea viitoare → 3 sugestii, cele mai devreme, ordonate, și `WARN` „assistant fallback” · P1 → 403; fără token → 401 · tabelul comparației completat din cele 20 de cereri.

III. Evenimente și servicii

Rezervarea trece de la „cine cheamă pe cine” la „cine ascultă ce”. Serviciile noi de aici intră în profilurile compose `messaging`, `tracing` și `logs`, care nu sunt în `COMPOSE_PROFILES` pe EC2: deploy-ul automat nu le pornește acolo. Rulează local și pe instanța de demo (minim 8 GB: Elasticsearch ia singur 1–2 GB), pornită pentru verificare și susținere.

F13 Rezervarea anunță, ceilalți reacționează

client · furnizor

La fiecare rezervare și anulare, clientul și furnizorul primesc o notificare în aplicație. Notificarea nu se mai face din metoda de rezervare: rezervarea anunță un eveniment, iar cine are treabă ascultă.

Pornești de la: `createBooking`, `cancelBooking` (F6), contoarele de rezervări (F8), `slot-mate-ops/docker-compose.yml` · **nou** Kafka (`apache/kafka`, un nod, KRaft, în compose, profilul `messaging`)

1. **Bază.** `V9__notifications.sql`, cu UQ `(event_id, recipient_id)`.
2. **Model/repository.** `Notification` și `NotificationRepository`.
3. **Evenimente.** `spring-boot-starter-kafka` în `slot-mate-api` (+ `org.testcontainers:testcontainers-kafka` pentru test). Rezervarea și anularea publică un eveniment Spring; un `@TransactionalEventListener(AFTER_COMMIT)` îl trimite în Kafka: `booking.created` / `booking.cancelled`, cheia = slotId, payload {eventId, bookingId, slotId, offeringName, providerName, startTime, clientId, clientEmail, providerUserId, providerEmail, cancelledBy}. Kafka căzut → `ERROR` + `slotmate.events.publish.failed`, iar rezervarea rămâne făcută. Publicarea pornește doar cu `EVENTS_ENABLED=true` (local și demo); pe EC2, unde profilul `messaging` nu rulează, `.env` are `false`, altfel fiecare rezervare ar scrie `ERROR`.
4. **Ascultători, tot în monolit.** `NotificationListener` (grupul `slot-mate-notifications`): o notificare pentru client, una pentru furnizor. Contoarele din F8 rămân în service: pe EC2 nu există Kafka, iar Grafana de acolo trebuie să le vadă mai departe. `autoStartup` din `NOTIFICATIONS_LISTENER_ENABLED` (`false` pe EC2).
5. **Idempotență.** Același eveniment de două ori nu dublează nimic: UQ-ul + prinderea `DataIntegrityViolationException`.
6. **Intrare.** `GET /api/notifications/me` (contractul trece la 1.6.0). **Permisiuni:** `NOTIFICATION_VIEW`, fiecare doar ale lui.
7. **Test cu login real.** Kafka din Testcontainers. Rezervă → notificările apar (Awaitility); același eveniment trimis de două ori → tot 2.
8. **Log / metrică.** `eventId` și `bookingId` în MDC în listener; lag-ul consumatorilor în Grafana.
9. **Ops (profilul `messaging`).** `kafka` cu volum și healthcheck; `kafka-init` creează cele două topicuri la pornire, din container, nu din cod; Kafka UI (kafbat) ca să vezi topicurile și offset-urile, doar local și la demo (din F16, în spatele lui oauth2-proxy).

Reguli

- Evenimentul pleacă doar după commit; o rezervare respinsă (409) sau un rollback nu produce eveniment.
- Un eveniment procesat de două ori = același rezultat. Consumatorul repornit preia de la ultimul offset confirmat, fără pierderi.

Gata când: local sau pe demo: Ana rezervă la P1 → `/api/notifications/me`: Ana 1, P1 1 · același `booking.created` retrimis cu `kafka-console-producer` → tot 1 și 1 · Ana anulează → 2 și 2 · API repornit cu `NOTIFICATIONS_LISTENER_ENABLED=false`, 3 rezervări (3 × 201), apoi cu `true` → Ana 5, P1 5.

F14 Notificările pleacă din monolit

client · furnizor

Clientul primește pe email confirmarea, anularea și un reminder cu 24 de ore înainte; furnizorul află pe email de fiecare rezervare nouă sau anulată. Dacă serverul de email cade, rezervările merg mai departe.

Pornești de la: `NotificationListener`, `Notification`, `GET /api/notifications/me` și evenimentele din F13 · **nou** al doilea serviciu (`slot-mate-notifier`); topicurile = singurul contract între servicii

1. **Contract.** `slot-mate-docs/events.yaml` (AsyncAPI): cele două topicuri și payload-ul lor. E tot ce știe notifier-ul despre API.
2. **Bază.** `notifier_db`, cu Flyway-ul lui: `V1__notifications` (`notifications` + `reminders`). În API, `V10__drop_notifications`.
3. **Proiect.** `slot-mate-notifier/`: Boot 4.1.2, Java 25, `spring-boot-starter-kafka`, `spring-boot-starter-mail`. Nu știe de users: destinatarii vin din eveniment.
4. **Service.** `booking.created`: confirmare către client, anunț către furnizor, reminder `PENDING` la startTime – 24h. `booking.cancelled`: anunț la amândoi, reminder `CANCELLED`. Un `ReminderJob`, la 5 minute, trimite reminder-ele scadente și le marchează `SENT`. Consumatorii notifier-ului: grupul `slot-mate-notifier`, `spring.kafka.consumer.auto-offset-reset=latest`.
5. **Intrare.** `GET /api/notifications/me` se mută aici, cu același JWT (același `JWT_SECRET` și o copie a `JwtAuthFilter`-ului din F2 pe claim-ul `permissions`, fără o copie a hărții; `NOTIFICATION_VIEW`); din API dispare, deci contractul API trece la 2.0.0.
6. **Idempotență.** UQ `(event_id, recipient_email, type)` și `reminders.booking_id` unic.
7. **Ops.** Mailpit în compose ca SMTP de test (UI pe 8025, doar local). Dockerfile, serviciu în compose, job în Actions filtrat pe `slot-mate-notifier/**`. Notifier-ul și Mailpit intră în profilul `messaging`: job-ul împinge imaginea, dar pe EC2 `up -d` nu le pornește. Local și la demo, nginx trimite `/api/notifications/` la notifier.
8. **Log / metrică.** `eventId` în MDC; `notifier.emails.sent{type}` și `notifier.emails.failed`. Un email eșuat se reîncearcă, fără să blocheze consumatorul.

Reguli

- API-ul fără notifier merge: rezervările se fac, evenimentele așteaptă în topic. Fiecare email pleacă o singură dată, chiar dacă evenimentul vine de două ori.
- Reminder doar pentru rezervările încă active, cu 24h înainte; anularea îl oprește.

Gata când: local sau pe demo: pornești notifier-ul o dată (grupul își confirmă offset-urile), îl oprești; apoi Ana rezervă un interval peste 23h → 201 · pornești notifier-ul → în Mailpit 2 emailuri; în ≤ 5 minute reminder-ul → 3 · interval peste 3 zile (+2 → 5), anulat (+2 → 7), fără reminder · primul `booking.created` retrimis → tot 7 · în baza API-ului, tabela `notifications` nu mai există.

F15 Urmăresc o rezervare prin tot sistemul

(profesorul, la susținere) · administrator

Pornind de la id-ul unei rezervări sau de la runId-ul agentului, găsești în Kibana toate liniile de log ale fluxului, din toate serviciile, iar în Grafana (Tempo) un singur drum: cine a chemat pe cine și cât a durat fiecare pas.

Pornești de la: logurile JSON cu MDC (F7), metricile și Grafana (F8), `slot-mate-agent` (F11–F12), `slot-mate-notifier` (F14), profilurile compose (F5b) · **nou** OpenTelemetry + Tempo (trace-uri) + ELK: Elasticsearch, Kibana, Filebeat (loguri)

1. **Bază.** Niciuna.
2. **Tracing.** `spring-boot-starter-opentelemetry` (Boot 4.1.2; aduce bridge-ul OTel, exportatorul OTLP și auto-configurarea) în cele trei aplicații, `management.tracing.sampling.probability=1.0` local și la demo, `management.opentelemetry.tracing.export.otlp.endpoint` spre colector. `traceId` și `spanId` intră în MDC, lângă runId, eventId și bookingId.
3. **Propagare.** `traceparent` trece prin headerele Kafka (observation pe `KafkaTemplate` și pe listener) și prin apelurile HTTP ale clientului MCP și ale login-ului. Clientul MCP folosește `java.net.http.HttpClient`, neinstrumentat: în `slot-mate-agent`, un bean `McpSyncHttpClientRequestCustomizer` pune pe fiecare cerere spre `/mcp` headerul `traceparent` din contextul curent (`Tracer` + `Propagator` injectate), în același loc unde pui `Authorization: Bearer`. Login-ul trece prin `RestClient` construit din `RestClient.Builder` (instrumentat automat). În notifier, trimiterea emailului e înfășurată într-o `Observation` („notifier.email.send”), ca SMTP să apară ca span.
4. **Loguri în ELK (profilul `logs`).** Cele trei aplicații scriu JSON (`ecs`, din F7) și local, și la demo, cu `traceId` în fiecare linie. Filebeat citește fișierele JSON ale containerelor (`/var/lib/docker/containers`, driver-ul `json-file`, montat read-only), adaugă numele serviciului din metadatele

Docker, decodează linia aplicației și o trimite în Elasticsearch (un nod, heap 1 GB, fără port publicat). Kibana: un data view pe indexul logurilor. `filebeat.yml` în `slot-mate-ops/filebeat/`.

5. **Trace-uri (profilul `tracing`).** Colectorul OTel și Tempo în compose. Grafana primește sursa de date Tempo și, pentru drumul trace → loguri, sursa Elasticsearch.
6. **Test.** Un eveniment publicat poartă headerul `traceparent`; o cerere a clientului MCP poartă headerul `traceparent`.

Reguli

- Un singur `traceId` de la request-ul HTTP până la emailul trimis. Eșantionare 100% local și la demo.
- Elasticsearch, Kibana si Tempo nu publică porturi; până la F16, Kibana se deschide printr-un tunel ssh, din F16 prin oauth2-proxy. Pe EC2 nu rulează (profilurile `logs` și `tracing`).

Gata când: pe demo: Ana cere o sugestie și rezervă intervalul primit · în Kibana cauți bookingId → liniile din api și din notifier, toate cu același `traceId` → în Grafana (Tempo): `POST /api/bookings` (api) → send `booking.created` → consume (notifier) → SMTP · `runId`-ul agentului în Kibana → `traceId` → agent → `/mcp getOccupancy` (api) → `proposeSlotPlan` (api) · toate trei serviciile apar în Tempo · în Kibana, filtrul pe serviciul notifier → doar liniile lui.

IV. Platforma, ca la cars

Aceleași aplicații, puse în spatele unei platforme: un singur login pentru oameni și pentru agent (Keycloak) și o singură poartă spre aplicații (Kong). Tot în docker compose, pe EC2 (profilurile `auth` și `edge`), nimic din Kubernetes: realm-ul, `kong.yml` și rețelele Docker țin locul resurselor de cluster.

F16 Un singur login pentru toată platforma

orice utilizator · administrator · agent

Oamenii se loghează la Keycloak, nu la API; cu același cont, administratorul deschide Grafana, Kibana și Kafka UI, iar ceilalți nu. Agentul primește token ca aplicație, fără parola unui om. Tabelul de permisiuni nu se schimbă: se mută doar locul în care stă.

Pornești de la: `AuthService`, `AuthTestHelper`, `RolePermissions`, `JwtAuthFilter`, `User`, `UserRepository`, `ApplicationRunner`-ul de admin (F2), contul `AGENT` din același runner (F10), clientul MCP și login-ul din `slot-mate-agent` (F11), validarea JWT din asistent (F12) și din notifier (F14), `DemoDataSeeder` (F8), nginx (F6), Grafana (F8), Kibana (F15), Kafka UI (F13) · **nou** Keycloak (realm în Git, resource server, client credentials, oauth2-proxy)

1. **Bază.** `V11__users_keycloak.sql`: din `users` pleacă `password_hash` (parolele stau doar în Keycloak) și `role` (rolurile stau în realm); `users.id` = `sub`-ul din Keycloak. Keycloak are Postgres-ul lui, `keycloak-db`, cu volum; în el nu scrii nimic de mână.
2. **Realm în Git.** `slot-mate-ops/keycloak/realm-slot-mate.json`, importat de container cu `start --import-realm` (doar dacă realm-ul nu există). Rolurile de realm = cele 13 permisiuni; `CLIENT`, `PROVIDER`, `ADMIN`, `AGENT` = roluri compuse, exact tabelul de sus; un mapper pune rolurile în claim-ul `permissions`. Grupul `/admins` (adminul). Clienți: `slot-mate-app` (public: Swagger cu PKCE, login-ul din teste și din verificări), `slot-mate-ui` (public, authorization code + PKCE `S256` obligatoriu, redirect și web origin doar `https://app.<domeniu>` și, local, `http://localhost:5173`; îl folosește UI-ul din U5, dar intră acum, pentru că realm-ul se importă doar o dată), `slot-mate-api` (confidențial, cont de serviciu doar cu `manage-users`, pentru register), `slot-mate-agent` (confidențial, client credentials, cont de serviciu cu rolul `AGENT` și email fix), `oauth2-proxy` (confidențial, cu claim-ul `groups`). Parolele și secretele din JSON sunt placeholder-e (`${ADMIN_PASSWORD}` …) luate din `.env`. Token valabil 3600 s, ca în F2. `KC_HOSTNAME` fix pe `https://auth.<domeniu>`, ca tokenul să aibă același `iss` oricum ar fi cerut, din internet sau din rețeaua compose.
3. **Model/repository.** `User` fără `passwordHash` și fără `role`. `Role` și `RolePermissions` dispar din API; enum-ul `Permission` rămâne (șirurile din `hasAuthority`), iar un test unitar citește realm-ul din Git și cere ca fiecare valoare să existe acolo ca rol.
4. **Service: login iese, register devine delegat.** `AuthService.login` și `POST /api/auth/login` dispar: tokenul îl dă Keycloak. `POST /api/auth/register` rămâne, cu aceleași reguli (doar `CLIENT` / `PROVIDER`, email unic, parolă ≥ 8): creează contul în Keycloak prin Admin API, cu rolul ales, apoi rândul din `users` cu id-ul primit; dacă al doilea pas pică, contul din Keycloak se șterge (nu rămân conturi pe jumătate). Adminul și agentul vin din realm: la prima lor cerere, API-ul le creează rândul din `users` din claim-uri (`sub`, `email`, `name`), ca FK-urile din `proposal_runs` și `slot_proposals` să rămână valide. Cele două `ApplicationRunner`-e dispar. `DemoDataSeeder` își face conturile prin același register. UI-ul din U1–U4 se loghează prin `POST /api/auth/login`, care dispare aici: F16 și U5 se fac împreună, U5 imediat după F16, ca UI-ul de pe EC2 să nu rămână nicio zi fără login.
5. **Intrare: trei resource servere pe același issuer.** api, `slot-mate-agent` (asistentul) și notifier-ul: `issuer-uri` = adresa publică a realm-ului, `jwk-set-uri` = adresa din rețeaua compose (`http://keycloak:8080/…`). `JwtAuthFilter` și `JwtService` din F2 pleacă din toate trei: tokenul îl validează `oauth2-resource-server`, iar un `JwtAuthenticationConverter` cu `JwtGrantedAuthoritiesConverter` pe claim-ul `permissions`, fără prefix, face aceleași authorities. `hasAuthority` nu se schimbă. `JWT_SECRET` dispare. springdoc: `bearerAuth` devine OAuth2 authorization code + PKCE pe `slot-mate-app`. Contractul API trece la 3.0.0 (login scos).
6. **Agentul pe client credentials.** Planificatorul (F11) nu mai cheamă `/api/auth/login`: cere tokenul la Keycloak cu `slot-mate-agent` și secretul lui, și cere altul înainte să expire. `AGENT_EMAIL` / `AGENT_PASSWORD` devin `AGENT_CLIENT_SECRET`. Asistentul (F12) trimite mai departe tokenul clientului, ca până acum.
7. **Permisiuni.** Matricea de sus, neschimbată, acum în realm. Contul de serviciu al agentului are `STATS_VIEW`, `SLOT_PROPOSE`, `SLOT_PROPOSAL_VIEW`, fără `SLOT_MANAGE` și fără `SLOT_PROPOSAL_APPROVE`. Rolul `manage-users` al lui `slot-mate-api` e al Keycloak-ului, nu al matricei, și nu ajunge în tokenul nimănui.
8. **oauth2-proxy pe UI-uri.** nginx cere, prin `auth_request`, aprobarea lui oauth2-proxy pentru `grafana.`, `kibana.` și `kafka-ui.<domeniu>`; intră doar grupul `/admins`. Portul 3000 al Grafana iese din Security Group. `/admin` al Keycloak-ului, doar de la IP-ul tău, în nginx.
9. **Test cu login real.** Keycloak în Testcontainers (`testcontainers-keycloak`), cu același `realm-slot-mate.json`. `AuthTestHelper`: register (delegat) → token de la Keycloak pe `slot-mate-app`; `adminToken()` se loghează pe adminul din realm. Toate testele de până acum îl folosesc mai departe, fără alte schimbări. Testele login-ului din F2 (HS256, `exp`, același detail) pleacă odată cu login-ul; în locul lor: token cu `iss` greșit → 401, token semnat cu altă cheie → 401, register cu pasul local căzut → contul nu rămâne în Keycloak. Testul de integrare al agentului ia tokenul prin client credentials.
10. **Log.** `INFO` la register cu userId; `WARN` la un register anulat. Parolele nu apar nicăieri.
11. **Livrare.** Keycloak, `keycloak-db` și oauth2-proxy în profilul `auth`, pe EC2: API-ul nu mai poate valida niciun token fără Keycloak. `COMPOSE_PROFILES=core,edge,obs,auth`; instanța EC2 trece la 4 GB, cu limite de memorie pe containere. DNS + certificate pentru `auth.` și `grafana.`; în `.env`: parola de admin a Keycloak-ului, `KEYCLOAK_DB_PASSWORD`, `ADMIN_EMAIL`, `ADMIN_PASSWORD`, `AGENT_CLIENT_SECRET`, `API_CLIENT_SECRET`, secretul și cheia de cookie ale oauth2-proxy. O singură dată: `docker compose down -v` (MySQL refăcut V1–V11, realm importat curat), cu profilul demo.

Reguli

- register: aceleași 400 / 409 ca în F2; prin register nu se creează `ADMIN` sau `AGENT` (400). Token valabil o oră.
- Token fără semnătura realm-ului, cu alt `iss` sau expirat → 401; cu token fără permisiune → 403, ca până acum.
- Grafana, Kibana și Kafka UI doar pentru `/admins`; un utilizator logat, dar din afara grupului → 403 de la oauth2-proxy.

Gata când: pe EC2, după `down -v` cu profilul demo, în aceeași zi: `https://auth.<domeniu>/realms/slot-mate/.well-known/openid-configuration` → 200, cu `issuer` = adresa publică · token pentru Ana de la Keycloak (`slot-mate-app`) → `permissions` conține `CLIENT` și cele 5 ale lui: `BOOKING_CREATE`, `BOOKING_VIEW`, `BOOKING_CANCEL`, `ASSISTANT_USE`, `NOTIFICATION_VIEW`; `exp - iat = 3600` · `/api/users/me` cu el → 200; un token HS256 din F2 → 401 · register Bob (`CLIENT`) → 201 și apare în Keycloak cu rolul `CLIENT`; cu `"role":"ADMIN"` → 400 · POST categorie: Bob → 403, adminul din realm → 201 · agentul repornit cu `AGENT_RUN_ON_STARTUP=true`: tokenul lui (client credentials) are `STATS_VIEW`, `SLOT_PROPOSE`, `SLOT_PROPOSAL_VIEW`, iar run-ul are tot 2 propuneri, ca în F11 (`ADD` marți 12:00, `REMOVE` marți 09:00) · tokenul agentului pe `/approve` → 403 · `https://grafana.<domeniu>` → login Keycloak: Ana → 403, adminul → Grafana · `curl -m 5 http://<ip>:3000` din afară → timeout · pe demo, Kibana și Kafka UI la fel: Ana 403, adminul intră.

F17 O singură poartă spre aplicații

orice client · agent · administrator

Orice cerere spre API, MCP, asistent sau notificări trece printr-o singură poartă: verifică tokenul, oprește abuzul, dă fiecărei cereri un id de corelare și numără tot. Aplicațiile nu se mai văd de nicăieri altundeva, nici măcar din nginx; agentul trece și el prin poartă.

Pornești de la: nginx (F6), realm-ul și issuer-ul Keycloak (F16), filtrul care pune `requestId` în MDC (F7), Prometheus și Grafana (F8), clientul MCP din `slot-mate-agent` (F11), rutele nginx spre asistent (F12) și notificări (F14), `client.ts` și login-ul prin Keycloak din UI (U5), `slot-mate-ops/docker-compose.yml` · **nou** Kong = API gateway (DB-less, `kong.yml` în Git) + rețelele Docker `edge` / `backend`

1. **Bază.** Niciuna.
2. **kong.yml în Git.** `slot-mate-ops/kong/kong.yml`, declarativ, Kong fără bază de date. Servicii: api, agent, notifier. Rute: una publică, doar pentru GET-urile publice din tabelul de sus, register, Swagger și `/v3/api-docs`, fără jwt; `/api` și `/mcp` → api, `/api/assistant` → agent, `/api/notifications` → notifier, toate cu jwt. Issuer-ul și cheia publică a realm-ului diferă local față de EC2: un container `kong-init` randează `kong.yml` din șablonul din Git cu valorile din `.env`, ca la `kafka-init`.
3. **jwt pe Keycloak.** Un consumer pentru realm, cu cheia publică RS256 (nu e secret) și cheia = `iss`; se verifică și `exp`. Dacă volumul `keycloak-db` se șterge, realm-ul primește altă cheie și o actualizezi în `.env`. Kong verifică doar tokenul; permisiunile rămân în service, ca până acum.
4. **rate-limiting.** 60 de cereri pe minut pe IP, pe rutele `/api` (publice și protejate); peste → 429. Kong are încredere în `X-Forwarded-For` doar de la nginx.
5. **correlation-id.** `X-Correlation-ID`, generat dacă lipsește, trimis la aplicație și întors clientului. Filtrul din F7 pune acest id ca `requestId` în MDC, deci același id apare în Kong, în api și, la demo, în Kibana.
6. **prometheus.** Metricile Kong pe portul de status (nepublicat), adăugate în `prometheus.yml`; în Grafana, un panou cu cereri pe rută și cod, 429-uri și latență.
7. **Rețele Docker.** Doar `edge`: nginx, oauth2-proxy, ui (fișiere statice, nu cheamă nimic din spate). În amândouă: Kong, Keycloak și UI-urile (Grafana, Kibana, Kafka UI), care au nevoie de ce e în spate. Doar `backend`: api, agent, notifier, MySQL, `keycloak-db`, Kafka, Prometheus, Elasticsearch, Filebeat, colectorul OTel, Tempo, Mailpit. nginx e doar în `edge`, deci nu mai ajunge la api: `api-locations.conf` (F6) trimite tot la `kong:8000`, deci și `api.<domeniu>`, și `app.<domeniu>/api/`, iar rutele spre asistent și notificări trec din nginx în `kong.yml`. nginx rămâne doar TLS + reverse proxy. Niciun serviciu în afară de nginx nu are `ports:`. Agentul cheamă `http://kong:8000/mcp`, nu `api:8080`.
8. **UI.** Cererile de date ale UI-ului rămân pe `app.<domeniu>/api/…`; se schimbă doar drumul din spate (nginx → Kong), deci în bundle tot nu există nicio adresă internă. În `client.ts`: 401 vine acum de la Kong, fără ProblemDetail, deci `apiFetch` nu mai presupune corp JSON și trimite la `login()`; orice mesaj de eroare arată și `X-Correlation-ID` din răspuns („Error id …"), același id ca în logul Kong și în logul api; 429 → „Too many requests, try again in a minute".
9. **Test.** În Actions, un job pe `slot-mate-ops/**`: `kong config parse` pe `kong.yml` randat și `docker compose config`. Testele aplicațiilor nu se schimbă (Kong nu e în ele). În testul Playwright al UI-ului, un caz nou: răspuns 401 fără corp (prin `page.route`) → pagina Keycloak.
10. **Log / metrică.** Logul de acces al Kong pe stdout, JSON, cu `X-Correlation-ID`. Kong intră în profilul `edge`, deci rulează și pe EC2.

Reguli

- Singurele porturi publicate: 80 și 443, ale lui nginx. Admin API-ul Kong nu e expus.
- Fără token, cu token stricat sau expirat pe o rută protejată → 401 de la Kong, cererea nu ajunge la aplicație (corpul e al Kong-ului, nu ProblemDetail). 403 vine tot din service.
- Peste 60 de cereri pe minut de la același IP → 429. Fiecare răspuns are `X-Correlation-ID`. Pe EC2, ruta de notificări dă 503: notifier-ul rulează doar local și la demo.
- UI-ul: 401 de la Kong → login din nou; 403 din service → `detail` ca până acum; 429 → „Too many requests, try again in a minute". UI-ul nu face cereri periodice.

Gata când: pe EC2: `docker compose ps` → doar nginx are porturi (80, 443) · `docker compose exec nginx wget -qO- -T 3 http://api:8080/api/categories` → eșec (nginx nu vede rețeaua `backend`) · `GET https://api.<domeniu>/api/categories` fără token → 200, cu `X-Correlation-ID`; același id în logul Kong și ca `requestId` în logul api · `/api/bookings/me` fără token → 401 de la Kong, iar în logul api nu apare niciun rând cu acel id; cu tokenul Anei → 200 · token cu semnătura modificată → 401 · 70 de GET-uri pe `/api/categories` trimise în primele 30 de secunde ale unui minut, de la același IP, fără alte cereri pe ruta publică în minutul acela → 60 × 200 și 10 × 429 · agentul repornit cu `AGENT_RUN_ON_STARTUP=true` → run creat ca în F16, iar în Prometheus `kong_http_requests_total` pe ruta `/mcp` crește · pe demo: `/api/notifications/me` cu tokenul Anei, prin Kong → 200 · UI-ul prin Kong: `https://app.<domeniu>` → Ana logată prin Keycloak → în DevTools, fiecare cerere de date merge la `app.<domeniu>/api/…` și are `X-Correlation-ID`, iar id-ul apare în logul Kong · Carla pe 10:00 deja rezervat → mesajul 409 cu „Error id …", același id în logul Kong și în logul api · de pe aceeași mașină, în minutul celor 70 de GET-uri, refresh pe catalog în browser → „Too many requests, try again in a minute" · Ana → Logout, apoi `/api/bookings/me` fără token → 401 de la Kong · în Grafana, panoul Kong arată cererile UI-ului pe `/api`.

V. Interfața — React (slot-mate-client)

Ce folosesc oamenii: o aplicație React în `slot-mate-client/`, crescută în cinci pași. Fiecare U se face imediat după feature-ul de backend scris în titlu și cheamă doar endpoint-uri care există deja. Forma e aceeași ca la F, cu pașii UI-ului: rute → `src/api/` → pagini → erori → ce vede fiecare rol → test Playwright cu login real → livrare. Codul și textele din UI în engleză; un singur stil: componente funcții, funcții nominale exportate, fișiere flat per resursă în `src/api/`, TypeScript fără clase, fără `any` și fără utility types.

U1 Catalogul și contul, după F5c

vizitator · client · furnizor

Oricine deschide aplicația vede categoriile, caută furnizori după categorie, oraș sau amândouă și deschide profilul unui furnizor. Își face cont, se loghează și își vede contul; o eroare de la API apare ca mesaj clar, nu ca pagină albă.

Pornești de la: folderul `slot-mate-client/` (gol, doar `.gitkeep`), `openapi.yaml` 1.2.0, `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/users/me` (F2), `GET /api/categories` (F1), `GET /api/providers`, `GET /api/providers/{providerId}` (F3), `slot-mate-ops/docker-compose.yml` și profilurile, workflow-ul din F5b · **nou** React + Vite + TypeScript, servit static de nginx-ul din imagine (+ Playwright pentru un test e2e de fum)

1. **Proiect.** Șablonul `react-ts` al Vite în `slot-mate-client/`, Node 24 LTS, `react-router` pentru pagini; versiunile fixe în `package-lock.json`, verificate pe npm în ziua în care le adaugi. Local, `vite.config.ts` trimite `/api` la `http://localhost:8080` (`server.proxy`, moștenit și de `vite preview`): browserul vede o singură origine, deci API-ul nu are nevoie de CORS, nici acum, nici mai târziu.
2. **Rute.** `/` (categorii + căutare), `/providers/:providerId`, `/register` (rolul: Client sau Provider), `/login`, `/me`. O pagină = o componentă funcție, un fișier în `src/pages/`.
3. **`src/api/`, flat per resursă.** `client.ts`: `apiFetch<T>(path, options)` cu `type ApiOptions = { method?: "GET" | "POST" | "PUT" | "DELETE"; body?: unknown }`, scrisă liniar: URL → headere (`Authorization: Bearer` dacă există token) → trimite → 204 fără corp → la eroare aruncă un `ApiProblem` (`status`, `detail`, `errors?` din ProblemDetail). Tokenul stă într-o variabilă a modulului, neexportată; se văd doar `setToken` și `clearToken`. `auth.ts`: `register`, `login`, `getMe`, `readPermissions(token)` (claim-ul `permissions` din payload, fără bibliotecă). `categories.ts`: `listCategories`. `providers.ts`: `searchProviders(categoryId?, city?)`, `getProvider`. DTO-urile în `types.ts`, copiate din `openapi.yaml`, folosite cu `import type`.
4. **Erori, într-un singur loc.** 401 pe o cerere care avea token (expirat după o oră) → `clearToken` și `/login` cu „Your session expired"; 401 la login → `detail`-ul în formular, fără redirect. 403 și 409 → `detail`-ul din ProblemDetail, afișat în pagină; 400 → mesajele din `errors` lângă câmpuri; API căzut → „Server unreachable".
5. **Ce vede fiecare rol.** Meniul se construiește din `permissions`: fără token, Catalog + Login + Register; logat, și My account. Ascunsul e doar comoditate: decizia rămâne în API (403).
6. **Test cu login real (Playwright).** Datele le pune testul prin API (adminul din `.env`-ul de test creează categoria, un furnizor își face profilul), apoi totul prin UI: register client → login → caută după categorie + oraș → 1 rezultat → profilul lui. Selectori cu `getByRole` sau `exact: true`: `getByText` simplu potrivește și subșiruri, fără litere mari/mici.
7. **Livrare.** `slot-mate-client/Dockerfile` în două etape: `node:24-alpine` (`npm ci`, `npm run build` = `tsc` + `vite build`) → `nginx:<tag fix>-alpine` doar cu `dist/` și `nginx.conf`: `try_files $uri /index.html` (fără ea, refresh pe `/providers/…` = 404) și, până la F6, `location /api/` → `http://api:8080`. În compose, serviciul `ui` (profilul `core`, deci pornește și pe EC2), imagine `ghcr.io/tomescuflavius/slot-mate-client:${UI_TAG:-latest}`, portul 80 publicat până la F6; Security Group: 80 deschis. `.github/workflows/ui.yml`, filtrat pe `slot-mate-client/**`: `test` (`npm ci`, build, MySQL + api de pe GHCR din compose, cu portul 8080 publicat doar în CI, `vite preview`, Playwright) → `image` → `deploy` (pașii din F5b, cu `UI_TAG`).

Reguli

- Tokenul stă doar în memorie, nu în `localStorage`: refresh pe pagină = login din nou. Parola nu se păstrează nicăieri după login.
- UI-ul nu calculează reguli: ordinea și filtrele vin din API. Textele în engleză; pe mobil, la 390px, fără derulare laterală.

Gata când: pe EC2, `http://<ip>/`, cu datele din F1–F3: fără login, 3 categorii în ordinea Barber, Frizerie, Sala de sport · Frizerie → 2 (P1, P3) · Cluj → 2 (P1, P2) · Frizerie + Cluj → 1 (P1) · click pe P1 → profilul: Frizerie, Cluj, adresa · `/providers/<UUID inexistent>` → mesajul 404, nu pagină albă · register Bob (`CLIENT`) în UI → login → My account: Bob, `CLIENT` · același email din nou → mesajul 409 · parolă de 7 caractere → mesajul de la câmpul parolei (400) · parolă greșită → mesajul 401 în formular · refresh pe `/me` → pagina de login · în Actions, `ui.yml` trece cele 3 joburi, cu testul Playwright verde · `docker compose ps` → ui cu tag-ul = SHA-ul commit-ului.

U2 Rezerv un loc, după F6

client · furnizor

Clientul alege un serviciu al furnizorului și o zi, vede intervalele libere, rezervă unul cu o notă și îl regăsește în „My bookings", de unde îl poate anula. Dacă altcineva i-a luat intervalul între timp, află asta clar și vede lista proaspătă.

Pornești de la: `apiFetch`, `auth.ts`, paginile din U1, `GET /api/providers/{providerId}/offerings` (F4), `GET /api/offerings/{offeringId}/slots` (F5), `POST /api/bookings`, `GET /api/bookings/me`, `DELETE /api/bookings/{bookingId}` (F6), `app.<domeniu>` pe nginx (F6) · fără unealtă nouă

1. **Rute.** Serviciile apar pe `/providers/:providerId`; `/offerings/:offeringId?date=…` (intervalele unei zile); `/bookings` (My bookings).
2. **`src/api/`.** `offerings.ts`: `listProviderOfferings`. `slots.ts`: `listSlots(offeringId, from, to, available)`. `bookings.ts`: `createBooking(slotId, note)`, `listMyBookings`, `cancelBooking`.
3. **Pagini.** Ziua aleasă devine `from = to`, cu `available=true`; selectorul de dată pornește de azi. Click pe interval → formular cu nota (≤ 255) → Confirm → `createBooking`. My bookings: data, ora, serviciul, furnizorul și nota, în ordinea din API; Cancel cere confirmare.
4. **Erori.** 409 → `detail`-ul („Slot … is already booked") lângă interval, apoi lista se reîncarcă din API. 400 pe notă → mesajul din `errors.note`.
5. **Ce vede fiecare rol.** Book și My bookings doar cu `BOOKING_CREATE`; furnizorul vede intervalele, fără buton (rezervările firmei vin în U3).
6. **Test cu login real (Playwright).** Două contexte de browser, doi clienți pe aceeași zi: primul rezervă 10:00 → 201; al doilea apasă pe 10:00 → mesajul 409, iar lista lui scade cu unul. Anularea → intervalul revine.
7. **Livrare.** Același `ui.yml`; de acum UI-ul e pe `https://app.<domeniu>`, prin nginx-ul din F6.

Reguli

- După orice scriere, lista se reîncarcă din API, nu se modifică local: sursa de adevăr e API-ul. Butonul de Confirm se blochează cât timp cererea e în zbor (un click = o rezervare).

Gata când: pe EC2, `https://app.<domeniu>`: P1 publică prin Swagger o zi nouă D2, tot nu marți, 09:00, 10:00 și 11:00 pe „Men's haircut" · Ana deschide P1 → 1 serviciu („Men's haircut", 30 min, 45.50; „Beard trim" a fost retras în F4) → D2 → 3 libere: 09:00, 10:00, 11:00 · Bob deschide aceeași zi · Ana rezervă 10:00 cu nota „First visit" → în My bookings apare D2 10:00, „Men's haircut", P1, nota · Bob, fără reîncărcare, apasă pe 10:00 → mesajul 409, iar lista lui devine 09:00, 11:00 · Ana anulează → dispare din My bookings → Bob reîncarcă → 3 libere → rezervă 10:00 → apare la el · logat ca P1: intervalele se văd, fără buton Book · `ui.yml` verde.

U3 Firma mea, după F10

furnizor

Furnizorul își administrează firma din aplicație: profilul, serviciile, intervalele, programul săptămânal, rezervările primite, ocuparea din ultimele săptămâni și propunerile agentului, pe care le aprobă sau le respinge.

Pornești de la: U1–U2, `POST` / `GET` / `PUT /api/providers/me` (F3), `POST /api/offerings`, `PUT` / `DELETE /api/offerings/{offeringId}` (F4), `POST /api/offerings/{offeringId}/slots`, `DELETE /api/slots/{slotId}` (F5), `GET /api/providers/me/bookings` (F6), `PUT` / `GET /api/offerings/{offeringId}/schedule` (F7), `GET /api/providers/me/occupancy` (F8), `GET /api/providers/me/slot-proposals`, `POST /api/slot-proposals/{proposalId}/approve` și `/reject` (F10) · fără unealtă nouă

1. **Rute.** Sub `/my-business`: profil, `services`, `services/:offeringId/slots?date=…`, `services/:offeringId/schedule`, `bookings`, `occupancy`, `proposals`.
2. **`src/api/`.** `providers.ts` + `getMyProvider`, `createMyProvider`, `updateMyProvider`; `offerings.ts` + `createOffering`, `updateOffering`, `deactivateOffering`; `slots.ts` + `createSlot`, `deleteSlot`; `schedules.ts`: `getSchedule`, `putSchedule`; `bookings.ts` + `listMyProviderBookings(from, to)`; `occupancy.ts`: `getMyOccupancy(weeks)`; `proposals.ts`: `listMyProposals`, `approveProposal`, `rejectProposal`.
3. **Pagini.** `getMyProvider` → 404 = formularul „Create your business". Ocuparea: un tabel zi × oră cu publicate, rezervate și rata, exact cum vin din API, plus totalurile; `weeks` 1–8. Propunerile: run-ul, apoi fiecare propunere cu tipul (`ADD` / `REMOVE`), ora, starea și motivul respingerii; Approve / Reject doar pe cele `PENDING` și valide.
4. **Erori.** Programul cu fereastră greșită → mesajul 400; fereastră suprapusă → 409; interval rezervat la ștergere → 409; propunere deja decisă → 409. Toate din `detail`, prin același `apiFetch`.
5. **Ce vede fiecare rol.** My business doar cu `PROVIDER_PROFILE_MANAGE`; Occupancy doar cu `STATS_VIEW`; Proposals doar cu `SLOT_PROPOSAL_APPROVE`.
6. **Test cu login real (Playwright).** Register `PROVIDER` → profil → serviciu → interval publicat → apare pe pagina publică a furnizorului; program cu fereastră greșită → mesajul 400.

Reguli

- UI-ul nu calculează ocuparea și nu decide valabilitatea unei propuneri: le arată pe cele din API. O aprobare trece prin `publishSlot` / `deleteSlot` din API, ca orice acțiune manuală.

Gata când: pe EC2, cu datele demo din F8 puse în aceeași zi (`down -v`, profilul demo; job-ul a publicat marțea viitoare 09:00…11:30): P1 (`p1@demo`) → My business: Frizerie, Cluj, „Men's haircut" 30 min, program marți 09:00–12:00 · Occupancy, 4 săptămâni: 09:00 0/4, 09:30 3/4, 10:00 4/4, 10:30 3/4, 11:00 4/4, 11:30 4/4 → 24 publicate, 18 rezervate, 75% · programul schimbat pe 09:00–12:10 → mesajul 400, programul rămâne · Ana rezervă din UI marțea viitoare 10:00 → la P1, Bookings pe marțea viitoare → 1 (Ana, 10:00) → P1 o anulează → la Ana dispare din My bookings · `AGENT` din MCP Inspector: `ADD` 12:00 și `REMOVE` 09:00 pe marțea viitoare → la P1, un run cu 2 propuneri valide → Approve pe `ADD` → `EXECUTED`, Reject pe `REMOVE` → `REJECTED` → marțea viitoare are 7 intervale (09:00…12:00), cel de la 12:00 cu sursa `AGENT` · P2 (`p2@demo`) → nicio propunere a lui P1 · Ana pe `/my-business/occupancy`, scris de mână → mesajul 403.

U4 Asistentul, după F12

client

Clientul îi scrie asistentului ce vrea, primește cel mult 3 intervale libere, fiecare cu motivul, și rezervă unul cu un click și o confirmare. Asistentul doar propune: rezervarea o face clientul, pe aceeași cale ca în U2.

Pornești de la: `createBooking` și formularul de confirmare (U2), `POST /api/assistant/suggestions` din `slot-mate-agent` (`openapi-agent.yaml`, F12), ruta `/api/assistant/` din `api-locations.conf` (F6, F12) · fără unealtă nouă

1. **Rute.** `/assistant`. În `vite.config.ts`, `/api/assistant` merge la agent, înaintea regulii pentru `/api` (ca în `api-locations.conf`).
2. **`src/api/`.** `assistant.ts`: `askAssistant(request: AssistantRequest): Promise<BookingSuggestion[]>`; tipurile din `openapi-agent.yaml`.
3. **Pagina.** Mesajul, plus câmpurile opționale (categorie, oraș, de la, până la). Fiecare mesaj trimis și răspunsul lui se adaugă în conversația de pe ecran; fiecare card arată furnizorul, serviciul, ora, prețul și motivul. Asistentul nu ține minte nimic între mesaje (F12 nu salvează nimic): fiecare mesaj pleacă întreg, cu câmpurile lui. Cât așteaptă (până la 20 s), Send e blocat și apare „Thinking…".
4. **Confirmarea.** „Book this" deschide formularul din U2 (nota) → Confirm → `createBooking`, aceeași funcție. 409 (intervalul a fost luat între timp) → mesajul pe card și sfatul de a întreba din nou.
5. **Erori.** 400 fără model și fără câmpuri → `detail`-ul lângă formular. 403 → mesajul.
6. **Ce vede fiecare rol.** Assistant doar cu `ASSISTANT_USE`.
7. **Test cu login real (Playwright).** În CI pornește și `slot-mate-agent` de pe GHCR, fără model: categorie + oraș + ziua → 3 carduri ordonate → Book pe primul → apare în My bookings.

Reguli

- Nicio rezervare fără click-ul și confirmarea clientului. UI-ul nu filtrează și nu reordonează sugestiile.

Gata când: pe EC2 (fără model, deci pe reguli), cu datele demo din aceeași zi: Ana scrie „A haircut next Tuesday morning in Cluj" cu Frizerie, Cluj, de la = până la = marțea viitoare → 3 carduri, cele mai devreme, în ordinea orei, aceleași 3 `slotId` ca un `curl` pe `/api/assistant/suggestions` cu aceleași câmpuri · Bob face aceeași cerere → aceleași 3 · Ana confirmă primul → 201 → în My bookings · Bob apasă Book pe același card → mesajul 409 · Ana trimite din nou → intervalul rezervat nu mai apare · un mesaj fără categorie și oraș → „categoryId or city is required when no model is available" · P1 nu are Assistant în meniu; pe `/assistant`, scris de mână → mesajul 403 · local, cu modelul pornit, doar textul → 1–3 carduri la o Frizerie din Cluj, înainte de 12:00.

U5 Login prin Keycloak, după F16

orice utilizator

Utilizatorul se loghează pe pagina Keycloak a platformei, nu într-un formular al aplicației, și rămâne logat la refresh; Logout îl scoate de tot. Contul nou se face tot din aplicație.

Pornești de la: `client.ts`, `auth.ts` și pagina de register (U1), clientul public `slot-mate-ui` din `realm-slot-mate.json` și register-ul delegat (F16) · **nou** OIDC în browser: authorization code + PKCE (`keycloak-js`)

1. **Config la build.** `VITE_KEYCLOAK_URL`, `VITE_KEYCLOAK_REALM`, `VITE_KEYCLOAK_CLIENT_ID` (`slot-mate-ui`): Vite le scrie în bundle la `npm run build`, nu le citește la rulare, deci local și EC2 = build-uri separate. În `ui.yml`, jobul `image` le primește ca build args din variabilele repo-ului (nu secrete: un client public nu are secret).
2. **`auth.ts`.** `keycloak-js` rămâne ascuns în modul; se exportă tot funcții nominale: `initAuth()` (`check-sso`, `pkceMethod: "S256"`), `login()` (redirect pe Keycloak), `logout()` (sesiunea Keycloak închisă, înapoi pe `app.<domeniu>`), `readPermissions()` din tokenul parsat. Pagina `/login`, `setToken` și apelul spre `/api/auth/login` dispar; register rămâne `POST /api/auth/register`, apoi `login()` cu emailul precompletat.
3. **`client.ts`.** Înaintea fiecărei cereri, tokenul se reînnoiește dacă mai are sub 30 s. 401 pe o cerere cu token (sesiunea Keycloak închisă) → `login()`, în locul paginii `/login`.
4. **Același drum pentru date.** Toate cererile de date merg tot la `app.<domeniu>/api/…` → nginx → api, ca din U1; cu Keycloak, UI-ul vorbește doar pentru login, logout și reînnoire. În bundle nu există nicio adresă internă.
5. **Test cu login real (Playwright).** Login pe pagina Keycloak (formularul ei) → My bookings. În Actions, pe build-ul de test, cu Keycloak din compose (profilul `auth`, același realm).
6. **Livrare.** Același `ui.yml`; imaginea de EC2 construită cu valorile de EC2.

Reguli

- În aplicație nu mai există câmp de parolă pentru login: parola se scrie doar pe `auth.<domeniu>`. Tokenul rămâne în memorie; refresh-ul păstrează login-ul prin sesiunea Keycloak, nu prin token pe disc.
- 401 → login din nou; 403 → `detail` ca până acum. UI-ul nu face cereri periodice.

Gata când: pe EC2, cu datele demo din F16 puse în aceeași zi: `https://app.<domeniu>` → Login → pagina Keycloak pe `auth.<domeniu>` → Ana (`ana@demo`) → înapoi în aplicație, cu meniul ei (Bookings, Assistant) · refresh pe `/bookings` → tot logată · în DevTools, fiecare cerere de date merge la `app.<domeniu>/api/…` , iar spre `auth.<domeniu>` doar login și reînnoire · marțea viitoare la P1: 6 libere (09:00…11:30) → Ana rezervă 10:00 → în My bookings · Carla se înregistrează din aplicație → 201 → apare în Keycloak cu rolul `CLIENT` → login pe pagina Keycloak → pe 10:00 → mesajul 409 · P1 → Occupancy: 24, 18, 75% · Ana → Logout → meniul fără Bookings; Back + refresh → tot delogată, iar `/api/bookings/me` fără token → 401.

Toate cele 24 (F1–F5, F5b, F5c, F6–F17 și U1–U5) fac parte din proiect, în ordinea: F1–F5, F5b, F5c, U1, F6, U2, F7–F10, U3, F11, F12, U4, F13–F16, U5, F17. Fiecare se sprijină pe cel dinainte: nu se sare. Ritmul real se vede abia la primul GET după contract (F1).
Versiunile. Spring Boot 4.1.1, apoi 4.1.2 (patch) din F9, singura schimbare de stack. Spring AI 2.0.2 până la susținere (2.1.0 e doar RC); linia 1.x e pentru Boot 3 și nu se folosește. UI-ul: Node 24 LTS, React, Vite, `react-router` , Playwright și `keycloak-js` cu versiuni fixe în `package-lock.json` , verificate pe npm în ziua în care le adaugi. Java 25 rămâne (Dockerfile `eclipse-temurin:25-jre` , CI pe temurin 25). Imaginile de platformă (Keycloak, Kong, oauth2-proxy, Elasticsearch / Kibana / Filebeat pe aceeași versiune, Tempo) cu tag fix, verificat în ziua în care le adaugi. Contractul crește: 1.3.0 (F7), 1.4.0 (F8), 1.5.0 (F10), 1.6.0 (F13), 2.0.0 (F14), 3.0.0 (F16, login scos); asistentul are `openapi-agent.yaml` , topicurile `events.yaml` .
Folderele. `slot-mate-client` primește UI-ul React (U1–U5). `slot-mate-mobile` și `slot-mate-db` nu intră în plan și se pot șterge: migrările stau în `slot-mate-api/src/main/resources/db/migration` . `slot-mate-ops` primește compose-ul și configurările din Git: `nginx/` (cu `api-locations.conf`), `grafana/` , `prometheus.yml` , `filebeat/` , `keycloak/realm-slot-mate.json` , `kong/kong.yml` , `.env.example` .
Doar Docker. `docker compose` cu profiluri, rețele Docker, healthcheck + `depends_on: service_healthy` , deploy prin ssh (`compose pull && up -d`). Niciodată Kubernetes, Helm sau ArgoCD.
EC2 și profilurile. Pe EC2, `COMPOSE_PROFILES` crește: `core` (F5b: MySQL, api; U1: ui, fișierele React servite static; F11: agent fără model, doar reguli și fallback) → `edge` (F6: nginx + certbot pe `api.` și `app.` ; F17: Kong) → `obs` (F8: Prometheus, Grafana) → `auth` (F16: Keycloak, `keycloak-db` , oauth2-proxy; instanța trece la 4 GB). `messaging` (Kafka, `kafka-init` , Kafka UI, notifier, Mailpit), `logs` (Elasticsearch, Kibana, Filebeat) și `tracing` (colectorul OTel, Tempo) nu încap: rulează local și pe o instanță de demo de minim 8 GB, pornită pentru verificare și susținere. Deploy-ul automat pornește doar profilurile din `.env` . Modelul AI rulează doar local sau la demo.
Secretele (`JWT_SECRET` până la F16, `ADMIN_*` , `AGENT_*` → `AGENT_CLIENT_SECRET` , `API_CLIENT_SECRET` , parolele Keycloak și `keycloak-db` , secretele oauth2-proxy, parola Grafana, `DEMO_PASSWORD` , o eventuală cheie de model) stau doar în `.env` , pe EC2 și local. Niciodată în repo, în compose, în `realm-slot-mate.json` (acolo doar placeholder-e) sau în `kong.yml` .